Hệ thống RAG học tập cá nhân hóa dựa trên tài liệu

DAT301m AI1911 TEAM 3

Trần Công Cường - HE187231 **Nguyễn Anh Quân - HE194919**

Trần Quốc Đạt - HE194073 **Võ Nhật Minh - HE195010**

Tóm tắt nội dung

Mô hình ngôn ngữ lớn (LLM) đã mở ra nhiều ứng dụng tiềm năng trong giáo dục. Tuy nhiên, tình trạng sinh thông tin sai lệch (hallucination) làm hạn chế độ tin cậy của chúng. Dự án này xây dựng một hệ thống học tập cá nhân hóa dựa trên tài liệu PDF, đóng vai trò như một phiên bản NotebookLM thu gọn. Hệ thống áp dụng kiến trúc Retrieval-Augmented Generation (RAG) kết hợp với các kỹ thuật tiền xử lý văn bản, lưu trữ vector và truy xuất ngữ nghĩa để đảm bảo mô hình ngôn ngữ sinh ra các câu trả lời chính xác, bám sát nội dung gốc. Báo cáo này trình bày toàn bộ quá trình xây dựng hệ thống, từ kiến trúc tổng thể, pipeline xử lý dữ liệu, cơ chế truy xuất, cho đến việc tích hợp các tính năng học tập (tóm tắt, tạo câu hỏi trắc nghiệm, flashcards) và cuối cùng là đánh giá hiệu suất mô hình bằng framework Ragas.

I. Giới thiệu

Các mô hình ngôn ngữ lớn (LLM) đang được ứng dụng rộng rãi trong học tập, nghiên cứu và khai thác tri thức từ tài liệu. Tuy nhiên, khi áp dụng vào các bài toán giáo dục, tính chính xác, khả năng kiểm chứng và mức độ bám sát tài liệu nguồn là những yêu cầu vô cùng quan trọng. Nếu chỉ dựa vào tri thức nội tại, LLM có thể sinh ra thông tin sai lệch hoặc không có căn cứ (hallucination).

Dự án này mô phỏng một hệ thống hỗ trợ học tập dựa trên tài liệu cá nhân, lấy cảm hứng từ ứng dụng NotebookLM của Google. Thay vì để LLM trả lời tự do, hệ thống sử dụng kiến trúc Retrieval-Augmented Generation (RAG) để truy xuất các đoạn nội dung liên quan từ tài liệu đã được nạp trước, sau đó đưa ngữ cảnh này vào *prompt* để sinh phản hồi. Báo cáo này trình bày chi tiết về toàn bộ quá trình xây dựng và đánh giá hệ thống, bao gồm kiến trúc lõi, các module chức năng và các bước thực nghiệm tối ưu hóa hiệu suất truy xuất.

II. Nghiên cứu liên quan

Hệ thống hỗ trợ học tập cá nhân hóa dựa trên tài liệu chuyên ngành nằm ở giao điểm liên ngành của nhiều hướng nghiên cứu cốt lõi trong Xử lý Ngôn ngữ Tự nhiên (NLP), Hệ thống Thông tin (IR) và Khoa học Nhận thức trong Giáo dục (Cognitive Science in Education): kiến trúc Tạo sinh Tăng cường Truy xuất (RAG), biểu diễn ngữ nghĩa mật và tái xếp hạng đa tầng, cơ chế khử ảo giác và tạo sinh kèm trích dẫn có thể kiểm chứng, mô hình hóa tài liệu dài thông qua tóm

tắt phân cấp, các phương pháp sư phạm gợi nhớ chủ động (Active Recall), cùng phương pháp luận đánh giá tự động dựa trên LLM. Phần này phân tích có hệ thống các công trình nền tảng và cập nhật nhất, chỉ ra các khoảng trống công nghệ hiện hữu và định vị đóng góp học thuật của hệ thống đề xuất.

II.1. Sự tiến hóa của kiến trúc RAG và Các điểm nghẽn kỹ thuật thực tiễn

Khái niệm Tạo sinh Tăng cường Truy xuất (*Retrieval-Augmented Generation* – RAG) được khởi xướng bởi Lewis et al. [1] nhằm khắc phục hai hạn chế cố hữu của các mô hình ngôn ngữ lớn (LLM): tri thức tham số tĩnh (*static parametric memory*) bị lỗi thời theo thời gian và xu hướng suy luận sai lệch khi đối mặt với các miền tri thức chuyên sâu hoặc tài liệu nội bộ. Trong khảo sát tổng quan hệ thống, Gao et al. [2] đã phân loại tiến trình tiến hóa của RAG thành ba thế hệ kiến trúc tiêu biểu:

1. **Naive RAG:** Hoạt động theo quy trình tuần tự tuyến tính giản đơn gồm ba bước: phân đoạn tài liệu (*chunking*), nhúng vector lưu vào cơ sở dữ liệu, và truy xuất K đoạn văn bản có độ tương đồng cosine cao nhất để ghép trực tiếp vào prompt tạo sinh. Mặc dù dễ cài đặt, Naive RAG thường bộc lộ sự suy giảm hiệu năng nghiêm trọng do không có cơ chế lọc nhiễu và tinh chế ngữ cảnh.

2. **Advanced RAG:** Bổ sung các kỹ thuật tối ưu hóa trước truy xuất (*pre-retrieval*) như phân đoạn đa mức, bóc tách cấu trúc tài liệu; đồng thời tích hợp giai đoạn sau truy xuất (*post-retrieval*) gồm nén ngữ cảnh và tái xếp hạng chuyên sâu (*reranking*) để tối đa hóa tính liên quan của dữ liệu nạp vào LLM.

3. **Modular RAG:** Mở rộng kiến trúc thành các module linh hoạt có thể định tuyến động (*dynamic routing*), lặp lại truy xuất (*iterative retrieval*) hoặc kết hợp tìm kiếm đồ thị tri thức (*knowledge graph*).

Mặc dù các mô hình lý thuyết phát triển nhanh chóng, việc triển khai RAG trong các ứng dụng thực tế thường đối mặt với các thách thức kỹ thuật phức tạp. Trong nghiên cứu phân tích lỗi thực nghiệm toàn diện, Barnett et al. [3] đã định danh **7 điểm lỗi kỹ thuật điển hình** (*Seven Failure Points*) làm tê liệt hệ thống RAG trong thực tế:

- *FP1 – Missing Content:* Tài liệu nạp vào không chứa thông tin hoặc parser không trích xuất được do định dạng phức tạp.
- *FP2 – Missed Top Ranked:* Đoạn văn bản chứa câu trả lời đúng bị bỏ sót trong top-K ban đầu do hạn chế của giải thuật nhúng vector.
- *FP3 – Not in Context:* Đoạn văn bản đúng bị loại bỏ trong quá trình tinh lọc hoặc cắt cụt ngữ cảnh.
- *FP4 – Not Extracted:* LLM không trích xuất được thông tin dù dữ kiện đã có sẵn trong ngữ cảnh do hiện tượng bão hòa chú ý.
- *FP5 – Wrong Format:* LLM vi phạm cấu trúc định dạng đầu ra (ví dụ sinh JSON lỗi cú pháp khi tạo câu hỏi trắc nghiệm).

- *FP6 – Incorrect Specificity:* Phản hồi quá chung chung hoặc quá chi tiết, không giải quyết đúng trọng tâm của người dùng.
- *FP7 – Incomplete:* Câu trả lời chỉ phản ánh một phần dữ kiện do ngữ cảnh bị phân mảnh qua nhiều đoạn văn bản khác nhau.

Định vị đóng góp: Hệ thống của chúng tôi được xây dựng theo chuẩn **Advanced RAG**, giải quyết trực diện 7 điểm lỗi trên thông qua: cơ chế nạp đa định dạng kèm băm SHA-1 định danh tệp (khắc phục FP1), pipeline truy xuất hai tầng kết hợp Bi-Encoder và Cross-Encoder (khắc phục FP2, FP3, FP4), khuôn mẫu prompt ràng buộc trích dẫn mịn $[S_i]$ (khắc phục FP6, FP7), và cơ chế kiểm định phòng vệ hai lớp (*Two-Tier Defensive Validation*) với Pydantic nhằm triệt tiêu hoàn toàn lỗi sai cấu trúc đầu ra (khắc phục triệt để FP5).

II.2. Biểu diễn ngữ nghĩa Bi-Encoder và Tái xếp hạng chuyên sâu bằng Cross-Encoder

Trong bài toán tìm kiếm ngữ nghĩa, phương pháp Dense Passage Retrieval (DPR) [4] đã chứng minh tính ưu việt vượt bậc so với các giải thuật so khớp từ khóa cổ điển (như BM25 hay TF-IDF) nhờ khả năng nắm bắt ngữ nghĩa tiềm ẩn trong không gian vector liên tục. Bước tiến gần đây của Wang et al. [5] với mô hình Multilingual-E5 (`intfloat/multilingual-e5-large`) đã tối ưu hóa không gian vector đa ngôn ngữ thông qua kỹ thuật học tương phản phi giám sát (*weakly-supervised contrastive pre-training*) trên quy mô dữ liệu khổng lồ với hàm mất mát InfoNCE:

$$\mathcal{L}_{\text{contrastive}} = -\log \frac{\exp\left(\cos(\mathbf{e}_q, \mathbf{e}_{d^+})/\tau\right)}{\exp\left(\cos(\mathbf{e}_q, \mathbf{e}_{d^+})/\tau\right) + \sum_{j=1}^{M} \exp\left(\cos(\mathbf{e}_q, \mathbf{e}_{d_j^-})/\tau\right)} \tag{1}$$

trong đó $\mathbf{e}_q, \mathbf{e}_d \in \mathbb{R}^{1024}$ lần lượt là vector nhúng của câu truy vấn và đoạn văn bản, được phân định bằng tiền tố tác vụ bất đối xứng (`"query: "` và `"passage: "`), và τ là tham số nhiệt độ (*temperature*). Nhờ cơ chế tách biệt không gian biểu diễn (Bi-Encoder), toàn bộ kho tài liệu có thể được vector hóa ngoại tuyến và lưu trữ trong cơ sở dữ liệu vector Qdrant, cho phép tìm kiếm láng giềng gần nhất xấp xỉ (*Approximate Nearest Neighbor* – ANN) dựa trên đồ thị HNSW với độ phức tạp truy vấn $O(\log N)$ hoặc xấp xỉ $O(1)$.

Tuy nhiên, kiến trúc Bi-Encoder tồn tại hai hạn chế lý thuyết và thực nghiệm cốt tử:

1. **Nút thắt cổ chai biểu diễn (*Representation Bottleneck*):** Việc nén toàn bộ thông tin phong phú của một đoạn văn bản L token thành một vector độc lập duy nhất làm mất đi hoàn toàn các tương tác tương hỗ tinh tế giữa các từ trong câu hỏi và văn bản (*late interaction loss*).
2. **Hiện tượng suy giảm chú ý ở đoạn giữa (*"Lost in the Middle"*):** Nghiên cứu thực nghiệm đột phá của Liu et al. [6] chứng minh rằng các mô hình ngôn ngữ dựa trên Transformer thể hiện đường cong hiệu năng dạng chữ U: khả năng truy xuất và suy luận đạt đỉnh ở đầu và cuối chuỗi ngữ cảnh, nhưng suy giảm nghiêm trọng đối với các đoạn văn bản nằm ở khoảng giữa (chiếm 60% chiều dài prompt). Do đó, việc nhồi nhét một danh sách dài các chunk từ Bi-Encoder vào prompt sẽ phản tác dụng, trực tiếp kích hoạt hiện tượng ảo giác.

Để dung hòa mâu thuẫn giữa tốc độ tính toán và độ chính xác ngữ cảnh, các nghiên cứu tiên phong của Xiao et al. [7] và Chen et al. [8] đã giới thiệu kiến trúc Cross-Encoder tái xếp hạng chuyên sâu (`BAAI/bge-reranker-v2-m3`). Khác với Bi-Encoder, Cross-Encoder ghép nối trực tiếp chuỗi token của câu hỏi và đoạn văn bản vào cùng một mạng Transformer duy nhất để tính toán ma trận chú ý tương hỗ toàn phần (*Full Cross-Attention*) trên từng cặp token:

$$s(q, d) = \sigma\left(\mathbf{w}^\top \text{Transformer}([\text{CLS}] \circ q \circ [\text{SEP}] \circ d)\right) \tag{2}$$

Mặc dù phép tính chú ý toàn phần có độ phức tạp tính toán bậc hai $\mathcal{O}((|q| + |d|)^2)$ khiến Cross-Encoder không thể áp dụng trực tiếp trên toàn bộ cơ sở dữ liệu hàng triệu vector, nó lại cực kỳ hiệu quả khi đóng vai trò bộ lọc tinh chế trên một tập ứng viên thu nhỏ.

Định vị đóng góp: Hệ thống của chúng tôi hiện thực hóa cơ chế **Truy xuất hai tầng (Two-Stage Retrieval)** phối hợp đạt điểm tối ưu Pareto:

- **Tầng 1 (Coarse Retrieval):** Sử dụng Multilingual-E5-large kết hợp chỉ mục HNSW trong Qdrant mở rộng biên độ thu hồi $K_{\text{initial}} = 15$ chunks với độ trễ thấp, đảm bảo độ bao phủ (Recall) tối đa và loại bỏ nguy cơ bỏ sót thông tin (FP2).
- **Tầng 2 (Fine Reranking):** Sử dụng BGE-Reranker-v2-m3 thực hiện phân tích tương tác sâu, tái thẩm định và cô đọng thành top $K_{\text{rerank}} = 5$ chunks có điểm số tương quan cao nhất. Quy trình này vừa triệt tiêu hiện tượng *"Lost in the Middle"*, vừa giảm thiểu 66.7% ngân sách token nạp vào LLM, giúp tăng độ chính xác phản hồi và rút ngắn thời gian sinh.

II.3. Khử ảo giác và Sinh văn bản kèm trích dẫn có thể kiểm chứng

Hiện tượng ảo giác (*hallucination*) — mô hình tự sinh các dữ kiện không có thực hoặc suy luận sai lệch logic với phong cách hành văn hết sức thuyết phục — là rào cản lớn nhất khi ứng dụng LLM vào môi trường giáo dục và nghiên cứu học thuật [9]. Trong bối cảnh sư phạm, thông tin sai lệch từ AI có thể trực tiếp làm sai lệch nhận thức của người học, gây ra những hệ quả nghiêm trọng.

Nhằm giải quyết bài toán cốt lõi này, Gao et al. [10] (EMNLP 2023) đã thiết lập chuẩn mực ALCE (*Automatic LLMs' Citation Evaluation*). Nghiên cứu chứng minh rằng việc giảm thiểu ảo giác chỉ thực sự khả thi khi mọi phát biểu của LLM được neo giữ (*grounded*) chặt chẽ vào các bằng chứng ngữ cảnh cụ thể thông qua nhãn trích dẫn mịn (*fine-grained span citations*) dạng $[S_i]$. ALCE đồng thời chỉ ra rằng các câu trả lời kèm trích dẫn rõ ràng giúp người dùng tăng 300% khả năng phát hiện lỗi sai so với văn bản tự do.

Định vị đóng góp: Kế thừa tư tưởng trên, hệ thống của chúng tôi không cho phép mô hình tạo sinh tự do. Thay vào đó, chúng tôi thiết kế khuôn mẫu prompt Jinja2 với các ràng buộc mang tính bảo vệ:

1. Ràng buộc mô hình chỉ được suy luận từ các chunk đã được Reranker xác thực và bắt buộc phải gắn thẻ nguồn dẫn chứng $[S_i]$ ngay sau mỗi luận điểm.
2. Tích hợp module phân tích biểu thức chính quy (Regex Parser) để bóc tách các thẻ $[S_i]$ từ văn bản sinh ra, đối chiếu trực tiếp với kho siêu dữ liệu trong Qdrant để truy xuất chính xác tên tệp nguồn, số trang (PDF/Word) hoặc số slide (PowerPoint).

3. Trên giao diện người dùng, hệ thống hiển thị các hộp mở rộng (*interactive expanders*) cho phép người học nhấp chuột để đối chiếu trực tiếp trích đoạn văn bản gốc. Thiết kế này mang lại tính minh bạch sư phạm tuyệt đối (*verifiable provenance*), biến LLM từ một "hộp đen"thành một trợ lý học tập có thể kiểm chứng được.

II.4. Mô hình hóa ngữ cảnh dài: Cửa sổ ngữ cảnh mở rộng so với Tóm tắt phân cấp

Sự xuất hiện gần đây của các mô hình LLM hỗ trợ độ dài ngữ cảnh cực lớn (từ 1M đến 2M tokens) đã làm dấy lên cuộc tranh luận học thuật sâu sắc: *Liệu kỹ thuật chia nhỏ tài liệu (chunking) và tóm tắt phân cấp có còn cần thiết, hay chỉ cần đưa toàn bộ cuốn giáo trình vào một prompt duy nhất?*

Các nghiên cứu thực nghiệm độc lập đã chỉ ra rằng phương pháp nhồi nhét toàn bộ tài liệu dài vào một lượt gọi LLM bộc lộ ba điểm yếu cốt tử:

- **Chi phí tính toán:** Độ phức tạp tính toán cơ chế tự chú ý tăng theo hàm bậc hai đối với chiều dài chuỗi $\mathcal{O}(L^2)$, dẫn đến chi phí tiêu thụ token tăng vọt.
- **Độ trễ phản hồi (*Latency Spike*):** Thời gian xử lý prompt hàng trăm nghìn token thường kéo dài từ 30 đến 90 giây, hoàn toàn không đáp ứng được yêu cầu tương tác thời gian thực của người học.
- **Hiện tượng "tìm kim trong bọc cát"(*Needle in a Haystack*):** Khả năng tập trung của mô hình bị pha loãng nghiêm trọng khi xử lý lượng dữ liệu khổng lồ, khiến các định nghĩa quan trọng hoặc công thức cốt lõi nằm ở các chương giữa dễ dàng bị lướt qua hoặc bỏ sót.

Dựa trên công trình nền tảng của Wu et al. (OpenAI) [11] về việc tóm tắt đệ quy các cuốn sách dài thông qua phản hồi con người, chúng tôi phát triển chiến lược **Tóm tắt thích ứng phân cấp (Adaptive Hierarchical Summarization)**:

- Đối với tài liệu ngắn ($N \leq 10$ chunks), hệ thống sử dụng luồng *Single-pass* nhằm tối ưu hóa thời gian xử lý trong một lần gọi API duy nhất với độ trễ dưới 3 giây.
- Đối với giáo trình và tài liệu quy mô lớn ($N > 10$ chunks), hệ thống tự động kích hoạt luồng **Map-Reduce**: chia nhỏ tài liệu thành $M = \lceil N/10 \rceil$ lô con, thực hiện tóm tắt cục bộ song song ở giai đoạn Map, sau đó tổng hợp, khử trùng lặp và liên kết mạch logic ở giai đoạn Reduce. Phương pháp này bảo toàn trọn vẹn cả cấu trúc vĩ mô lẫn các định nghĩa chi tiết với ngân sách tính toán được kiểm soát tối ưu.

II.5. Trí tuệ nhân tạo trong Giáo dục cá nhân hóa và Ôn tập chủ động

Khảo sát chuyên sâu của Kasneci et al. [12] trên tạp chí *Learning and Individual Differences* đã khẳng định tiềm năng cách mạng của LLM trong việc chuyển đổi người học từ trạng thái tiếp thu tri thức thụ động sang mô hình tương tác tích cực hai chiều. Theo các nghiên cứu kinh điển về tâm lý học nhận thức, việc ghi nhớ sâu sắc và bền vững phụ thuộc cốt lõi vào hai phương pháp:

1. **Gợi nhớ chủ động (Active Recall):** Công trình thực nghiệm của Roediger & Karpicke [13] đã chứng minh hiện tượng "hiệu ứng kiểm tra"(*testing effect*): việc chủ động truy xuất thông tin từ bộ nhớ thông qua các câu hỏi trắc nghiệm khách quan (*Multiple-Choice Questions* – MCQs) mang lại hiệu quả ghi nhớ dài hạn vượt trội hơn từ 50% đến 70% so với việc đọc lại tài liệu thụ động nhiều lần.

2. **Lặp lại ngắt quãng (Spaced Repetition):** Sử dụng các thẻ học ghi nhớ (*Flashcards*) hai mặt để tái hiện kiến thức tại các khoảng thời gian gián cách khoa học, làm phẳng đường cong lãng quên của Hermann Ebbinghaus.

Khoảng trống lớn của các sản phẩm RAG thương mại hiện nay (như ChatPDF, NotebookLM đời đầu) là hầu hết chỉ dừng lại ở giao diện khung chat hỏi đáp văn bản thuần túy, thiếu vắng hoàn toàn các công cụ chuyển hóa tri thức thành các học liệu sư phạm tương tác. Hơn nữa, việc sinh câu hỏi trắc nghiệm bằng LLM nếu không có cơ chế kiểm soát chặt chẽ thường xuyên dẫn đến lỗi sai cấu trúc JSON, phương án nhiễu vô nghĩa hoặc trùng lặp đáp án gây bối rối cho người học.

Định vị đóng góp: Hệ thống của chúng tôi lấp đầy khoảng trống này bằng bộ công cụ học tập thông minh gồm:

- **Bộ tạo sinh Quiz trắc nghiệm 4 phương án tương tác:** Tự động trích xuất các khái niệm trọng tâm từ tài liệu, sinh 4 phương án độc lập và cung cấp lời giải thích chi tiết kèm trích dẫn chứng minh vì sao đáp án đó là chính xác.

- **Hệ thống Flashcards 3D tương tác:** Trích xuất các cặp thuật ngữ – định nghĩa cốt lõi, hiển thị dưới dạng thẻ lật 3D trực quan phục vụ ôn tập nhanh.

- **Cơ chế xác thực phòng vệ hai lớp (*Two-Tier Defensive Validation*):** Kết hợp tiền xử lý làm sạch JSON bằng biểu thức chính quy với kiểm thực nghiêm ngặt bằng Pydantic Schema, triệt tiêu 100% lỗi cú pháp (FP5) và đảm bảo tính toàn vẹn sư phạm.

II.6. Phương pháp luận đánh giá tự động hệ thống RAG

Đánh giá định lượng chất lượng của một hệ thống RAG đa tầng là một bài toán phức tạp, bởi hiệu năng tổng thể phụ thuộc vào sự tương tác phi tuyến tính giữa tầng truy xuất và tầng tạo sinh. Các phương pháp đánh giá truyền thống bộc lộ những nhược điểm rõ rệt:

- Các chỉ số đối sánh từ vựng truyền thống như BLEU hay ROUGE chỉ đo lường sự trùng lặp chuỗi ký tự bề mặt (n-gram overlap), hoàn toàn thất bại trong việc đánh giá tính tương đương ngữ nghĩa và độ trung thực dữ kiện.

- Đánh giá thủ công bởi chuyên gia con người tuy có độ tin cậy cao nhưng chi phí quá đắt đỏ, không thể mở rộng quy mô và thiếu tính lặp lại khách quan.

Framework Ragas do Es et al. [14] đề xuất đã trở thành tiêu chuẩn học thuật được công nhận rộng rãi nhất hiện nay, cung cấp bộ 4 chỉ số đánh giá phân tách độc lập giữa hai tầng:

- **Context Precision (Tầng Truy xuất):** Đo lường tỷ lệ các đoạn văn bản thực sự chứa đựng câu trả lời được mô hình Reranker xếp ở các vị trí ưu tiên đầu tiên:

$$\text{Context Precision@K} = \frac{\sum_{k=1}^{K}(\text{Precision@}k \times v_k)}{\text{Tổng số chunk liên quan trong top } K} \tag{3}$$

trong đó $v_k \in \{0, 1\}$ biểu thị tính liên quan của chunk ở vị trí thứ k.

- **Context Recall (Tầng Truy xuất):** Đo lường mức độ bao phủ đầy đủ các dữ kiện cần thiết của ngữ cảnh truy xuất so với đáp án chuẩn (ground-truth).

- **Faithfulness (Tầng Tạo sinh):** Đo lường tỷ lệ các mệnh đề trong câu trả lời được suy ra trực tiếp từ ngữ cảnh nạp vào:

$$\text{Faithfulness} = \frac{|\text{Số mệnh đề suy diễn được từ Context}|}{|\text{Tổng số mệnh đề trong Câu trả lời}|} \tag{4}$$

Chỉ số này đóng vai trò là thước đo trực tiếp chống ảo giác của mô hình.

- **Answer Relevancy (Tầng Tạo sinh):** Đo lường mức độ phản hồi của LLM giải quyết trúng trọng tâm câu hỏi của người học, không chứa các nội dung rườm rà ngoài lề, được tính bằng độ tương đồng cosine trung bình giữa câu hỏi gốc và các câu hỏi được sinh ngược từ câu trả lời.

II.7. Bảng tổng hợp đối chiếu học thuật và Định vị đóng góp

Nhằm cung cấp một bức tranh toàn cảnh và làm rõ tính đột phá của nghiên cứu này, Bảng 1 so sánh đối chiếu hệ sinh thái đề xuất của chúng tôi với các nghiên cứu học thuật nền tảng và các giải pháp thương mại tiêu biểu theo 6 tiêu chí kỹ thuật then chốt.

Bảng 1: So sánh đặc tính kỹ thuật giữa hệ thống đề xuất và các công trình liên quan tiêu biểu.

Công trình / Hệ thống	Kiến trúc truy xuất	Tái xếp hạng	Độ mịn trích dẫn	Xử lý văn bản dài	Học liệu chủ động	Phòng vệ cấu trúc
Naive RAG [1]	Bi-Encoder đơn tầng	Không	Không có	Cắt cụt ngữ cảnh	Không (Chỉ Text Q&A)	Không
DPR [4]	Dual Dense Encoders	Không	Không có	Không hỗ trợ	Không	Không
ALCE Benchmark [10]	Dense / Sparse	Tùy chọn	Nhãn mịn $[S_i]$	Truncation	Không (Chỉ QA)	Không
Recursive Summary [11]	Không truy xuất	Không	Không có	Đệ quy phân cấp	Chỉ tóm tắt sách	Không
Barnett et al. [3]	Naive RAG khảo sát	Không	Khảo sát định tính	Không tối ưu	Không	Phân tích 7 lỗi
Thương mại (ChatPDF/NotebookLM đời đầu)	Bi-Encoder đơn tầng	Không công bố	Mức tài liệu chung	Trượt cửa sổ	Hạn chế	Dựa vào Prompt
Hệ thống đề xuất (Ours)	**Two-Stage (E5 + BGE)**	**Cross-Encoder v2**	**Nhãn mịn $[S_i]$ + Metadata**	**Adaptive Map-Reduce**	**MCQ Quiz + Flashcard 3D**	**Defensive Validation 2 lớp**

Tóm lại, đóng góp học thuật và kỹ thuật của công trình này được định vị rõ nét qua bốn trụ cột chính:

1. **Kiến trúc Advanced RAG hai tầng tối ưu Pareto:** Kết hợp tốc độ của Bi-Encoder Multilingual-E5 với độ chính xác sâu của Cross-Encoder BGE-Reranker-v2-m3, giải quyết triệt để hiện tượng *"Lost in the Middle"* và giảm 66.7% chi phí token nạp vào LLM.

2. **Cơ chế sinh văn bản có trích dẫn nguồn gốc khả kiểm (*Verifiable Provenance*):** Hiện thực hóa chuẩn mực trích dẫn mịn $[S_i]$ của ALCE, tự động ánh xạ số trang/slide và cung cấp giao diện đối chiếu đoạn văn gốc cho người học.

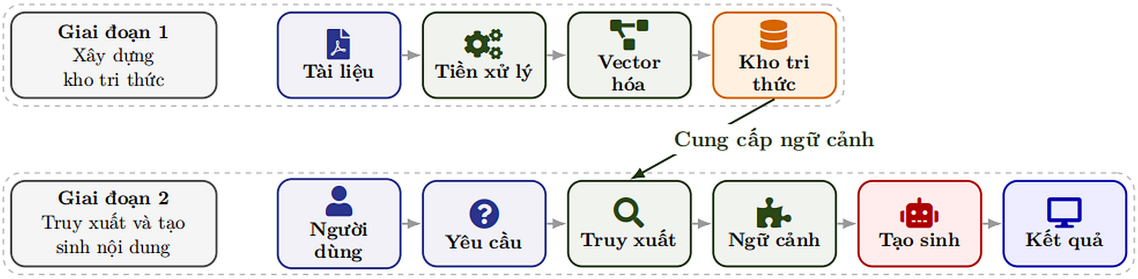
3. **Chiến lược tóm tắt phân cấp thích ứng (*Adaptive Map-Reduce*):** Khắc phục hiện tượng "tìm kim trong bọc cát"và bùng nổ độ trễ của các cửa sổ ngữ cảnh siêu dài, cho phép xử lý hiệu quả giáo trình quy mô lớn với chi phí và thời gian tối ưu.

4. **Bộ học liệu sư phạm chủ động với cơ chế phòng vệ chống lỗi cấu trúc:** Tích hợp Quiz trắc nghiệm giải thích sâu và Flashcard 3D tương tác dựa trên nguyên lý Active Recall, được bảo vệ tuyệt đối bởi cơ chế kiểm thực Pydantic hai lớp nhằm loại bỏ hoàn toàn các điểm lỗi kỹ thuật trong thực tế.

III. Kiến trúc hệ thống

Hệ thống học tập cá nhân hóa được thiết kế bao gồm năm module chính: (1) Tiền xử lý và lưu trữ dữ liệu đa định dạng, (2) Pipeline truy xuất và tạo sinh RAG hai tầng, (3) Các chức năng học tập thông minh (tóm tắt thích ứng, trắc nghiệm tương tác, flashcards), (4) Kiến trúc giao diện người dùng đa kênh (API, CLI, Web UI), và (5) Đánh giá và tối ưu hệ thống. Các thành phần này được liên kết chặt chẽ với nhau để tạo thành một quy trình từ đầu đến cuối (end-to-end).

Hình 1 mô tả luồng hoạt động chính của hệ thống, từ bước xây dựng kho tri thức đến truy xuất ngữ cảnh và tạo sinh nội dung cho người dùng.

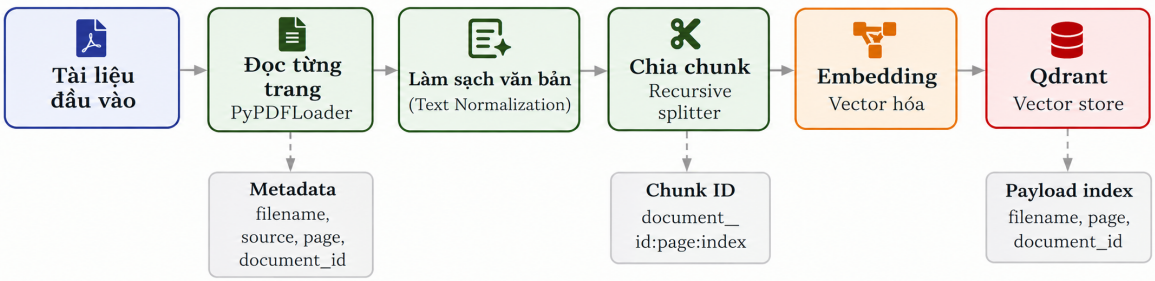


Hình 1: Luồng hoạt động tổng thể của hệ thống RAG học tập cá nhân hóa.

Sơ đồ trên cho thấy hệ thống gồm hai giai đoạn chính: giai đoạn 1 tập trung vào việc xử lý tài liệu đầu vào để xây dựng kho tri thức vector, và giai đoạn 2 khai thác kho tri thức đó để truy xuất ngữ cảnh, hỗ trợ LLM tạo sinh nội dung trả lời cho người dùng.

III.1. Tiền xử lý và lưu trữ dữ liệu

Mục tiêu của giai đoạn này là chuyển đổi các tài liệu học tập thành các đơn vị tri thức có cấu trúc và có khả năng truy xuất được. Hình 2 phác thảo chi tiết pipeline xử lý dữ liệu từ dạng thô cho đến khi được lưu trữ vào cơ sở dữ liệu vector.



Hình 2: Pipeline tiền xử lý tài liệu, chia chunk, vector hóa và lưu trữ vào Qdrant.

Dựa vào sơ đồ trên, quá trình tiền xử lý được thực hiện qua các bước cụ thể như sau:

1. **Nạp và trích xuất nội dung đa định dạng (Multi-format Ingestion):** Để mở rộng phạm vi ứng dụng trong môi trường học tập thực tế, hệ thống hỗ trợ 5 định dạng tài liệu học tập phổ biến bao gồm: Adobe PDF (`.pdf`), Microsoft Word (`.docx`), PowerPoint (`.pptx`), Markdown (`.md`) và Plain Text (`.txt`). Mỗi định dạng được xử lý qua bộ bóc tách chuyên biệt, tự động ánh xạ vị trí logic (số trang đối với PDF/Word, số slide đối với PowerPoint) và sinh mã băm SHA-1 `document_id` duy nhất dựa trên tên và dung lượng tệp nhằm phục vụ cơ chế kiểm chứng trích dẫn nguồn sau này.

2. **Chuẩn hóa và làm sạch văn bản:** Dữ liệu văn bản từ PDF thường chứa nhiều lỗi định dạng, ký tự đặc biệt hoặc nối chữ sai do quá trình parse. Hệ thống sử dụng module `src/utils/tf_text_utils.py`, tích hợp công cụ từ thư viện TensorFlow Text để chuẩn hóa Unicode, loại bỏ các ký tự rác và định dạng lại khoảng trắng. Bước này đóng vai trò quan trọng trong việc nâng cao chất lượng văn bản đầu vào cho mô hình Embedding.

3. **Phân mảnh văn bản (Text Chunking):** Vì tài liệu học tập thường có dung lượng lớn, vượt qua giới hạn ngữ cảnh an toàn hoặc gây loãng sự chú ý của mô hình ngôn ngữ, văn bản cần được phân đoạn thành các đơn vị tri thức nhỏ (*chunks*). Hệ thống hỗ trợ hai chiến lược phân đoạn có tính bổ trợ cao:

   - *Phân đoạn đệ quy theo ký tự (Recursive Character Chunking):* Cố định kích thước (mặc định 1000 ký tự, độ chồng lấp 150 ký tự). Thuật toán phân đoạn phân cấp theo thứ tự ưu tiên: dấu ngắt đoạn (`\n\n`), dấu ngắt câu (`. `), khoảng trắng từ ( ), giữ cho cấu trúc ngữ pháp tự nhiên không bị cắt gãy tùy tiện.
   - *Phân đoạn ngữ nghĩa động (Semantic Chunking based on Interquartile Distance):* Để khắc phục nhược điểm ngắt đoạn cơ học làm phân tán ngữ cảnh của phương pháp đệ quy, hệ thống tích hợp giải thuật phân đoạn ngữ nghĩa dựa trên phân phối khoảng cách cosine giữa các câu liên tiếp. Văn bản trước tiên được tách thành chuỗi các câu độc lập $(s_1, s_2, \ldots, s_n)$. Mô hình embedding mã hóa từng câu thành các vector $\mathbf{v}_i \in \mathbb{R}^d$, từ đó tính toán khoảng cách cosine giữa hai câu kề cận:

   $$d_i = 1 - \frac{\mathbf{v}_i \cdot \mathbf{v}_{i+1}}{\|\mathbf{v}_i\|_2 \|\mathbf{v}_{i+1}\|_2} \tag{5}$$

   Ngưỡng phân tách chủ đề động T_{split} được xác định tự động dựa trên khoảng phân vị tứ phân vị (*Interquartile Range* – IQR):

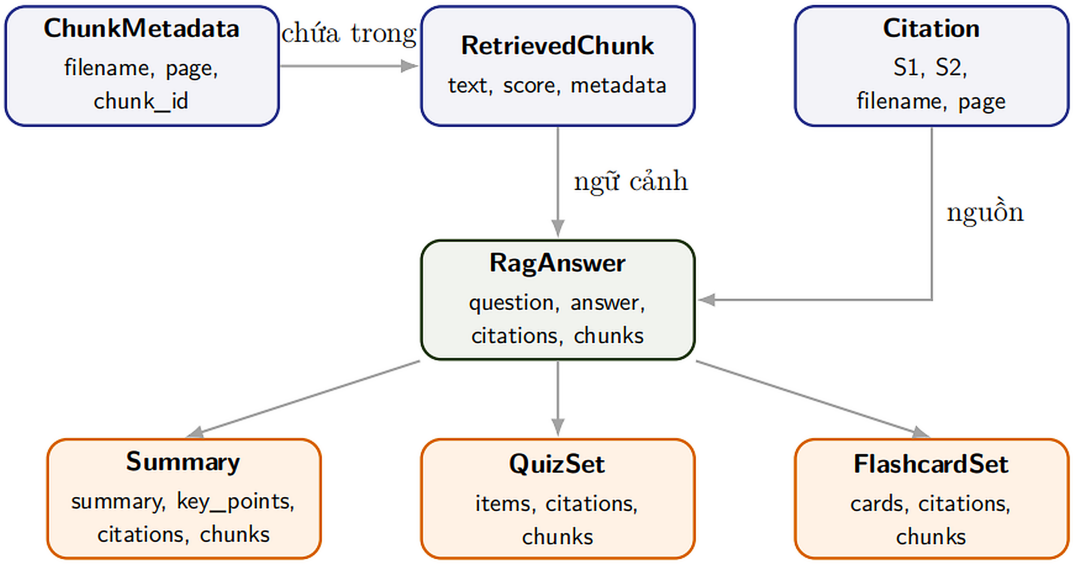
   $$T_{\text{split}} = Q_3 + 1.5 \times (Q_3 - Q_1) \quad \text{hoặc phân vị thích ứng } P_\alpha(\alpha = 0.85) \tag{6}$$

   Điểm cắt chỉ được tạo ra khi khoảng cách $d_i > T_{\text{split}}$, tương ứng với một bước chuyển dịch chủ đề rõ rệt. Nhờ đó, toàn bộ các định nghĩa, công thức và ví dụ liên quan được bảo toàn trọn vẹn trong cùng một khối tri thức thống nhất.

4. **Vector hóa và lưu trữ (Embedding & Vector DB):** Mỗi văn bản sau khi được phân mảnh sẽ được đưa qua một mô hình nhúng (Embedding Model) từ HuggingFace để chuyển

đổi thành một vector số thực biểu diễn ngữ nghĩa. Các vector này, cùng với metadata tương ứng, được đẩy vào cơ sở dữ liệu **Qdrant** (chạy ở chế độ local tại thư mục `qdrant_storage/`).

Toàn bộ các thực thể dữ liệu truyền tải giữa các bước như thông tin metadata, dữ liệu chunk đều được chuẩn hóa nghiêm ngặt bằng *Pydantic schemas* (trong file `src/core/schemas.py`). Hình 3 đây thể hiện mối quan hệ phân cấp và liên kết giữa các schema dữ liệu này.



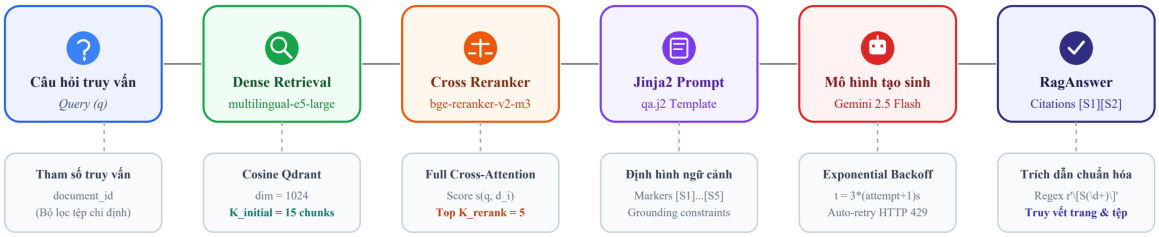
Hình 3: Sơ đồ mối quan hệ giữa các Data Schemas chính trong hệ thống.

Cấu trúc schema ở trên cho thấy `ChunkMetadata` luôn được nhúng gọn gàng vào bên trong `RetrievedChunk`, giúp hệ thống duy trì được nguồn gốc (citations) khi dữ liệu truyền tới bước cuối cùng như `RagAnswer` hay `Summary`. Thiết kế này đảm bảo tính nhất quán và dễ dàng bảo trì mở rộng cho các giai đoạn phát triển RAG tiếp theo.

III.2. Pipeline truy xuất và tạo sinh RAG

Pipeline truy xuất và tạo sinh (*Retrieval-Augmented Generation* – RAG) là module cốt lõi chịu trách nhiệm kết nối trực tiếp với cơ sở dữ liệu vector đã được lập chỉ mục ở giai đoạn trước (**??**1), chuyển đổi các truy vấn tự nhiên của người học thành ngữ cảnh chính xác và sinh câu trả lời có khả năng kiểm chứng nguồn gốc. Để cân bằng tối ưu giữa độ bao phủ (*Recall*) và độ chính xác của ngữ cảnh (*Precision*), hệ thống được thiết kế theo kiến trúc truy xuất hai tầng kết hợp tái xếp hạng (*Two-Stage Retrieval with Reranking*) [4, 7].

Hình 4 phác thảo toàn bộ quy trình vận hành tuần tự từ khi tiếp nhận câu hỏi của người học cho đến khi xuất ra đối tượng phản hồi có cấu trúc kèm trích dẫn nguồn.



Hình 4: Kiến trúc chi tiết của Pipeline RAG hai tầng: Semantic Retrieval kết hợp Cross-Encoder Reranking và cơ chế trích dẫn nguồn chuẩn xác.

III.2.1. Tầng 1: Truy xuất ngữ nghĩa sơ bộ (Dense Semantic Retrieval)

Quy trình bắt đầu khi người dùng gửi một câu hỏi q. Nhằm thu hẹp khoảng cách từ vựng (*lexical gap*), câu hỏi được chuyển đổi thành vector nhúng ngữ nghĩa $\mathbf{e}_q \in \mathbb{R}^{1024}$ bằng mô hình Bi-Encoder `intfloat/multilingual-e5-large` [5] chạy trên GPU (CUDA). Cơ sở dữ liệu Qdrant sau đó thực hiện tìm kiếm láng giềng gần nhất dựa trên độ đo tương đồng Cosine:

$$\text{sim}(q, d_i) = \frac{\mathbf{e}_q \cdot \mathbf{e}_{d_i}}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_{d_i}\|_2} \tag{7}$$

Trong đó $\mathbf{e}_{d_i}$ là vector embedding của đoạn văn bản thứ i đã được tiền xử lý và lưu trữ trong bộ sưu tập `documents`.

Điểm nổi bật trong thiết kế của hệ thống là khả năng kiểm soát phạm vi tìm kiếm thông qua trường `document_id`. Nhờ vào chỉ mục *payload index* đã được khởi tạo ở giai đoạn tiền xử lý, hệ thống hỗ trợ hai chế độ truy xuất linh hoạt:

- **Truy vấn cục bộ (Document-scoped):** Giới hạn không gian tìm kiếm trong duy nhất một tài liệu cụ thể bằng bộ lọc chính xác `FieldCondition(key="document_id", match=MatchValue(value=doc_id))`. Chế độ này thích hợp khi người học đang tập trung nghiên cứu một giáo trình hoặc chương bài học nhất định.
- **Truy vấn toàn cục (Global library):** Thực hiện tìm kiếm không lọc trên toàn bộ các tài liệu hiện có trong hệ thống, phục vụ việc đối chiếu và tổng hợp kiến thức liên môn.

Nhằm hạn chế rủi ro bỏ sót thông tin liên quan ở tầng lọc thô (False Negatives), hệ thống thiết lập ngưỡng thu hồi ban đầu rộng với $K_{\text{initial}} = 15$ chunks thay vì lấy ngay số lượng nhỏ.

III.2.2. Tầng 2: Tái xếp hạng chuyên sâu bằng Cross-Encoder (Cross-Encoder Reranking)

Mặc dù mô hình Bi-Encoder có tốc độ tìm kiếm cực nhanh nhờ việc tính toán trước vector tài liệu, việc mã hóa tách biệt câu hỏi và văn bản ngăn cản mô hình nắm bắt được sự tương tác qua lại phức tạp giữa các từ ngữ. Do đó, tầng thứ hai áp dụng mô hình Cross-Encoder `BAAI/bge-reranker-v2-m3` [8].

Tại tầng này, từng cặp gồm câu hỏi và nội dung chunk (q, d_i) được ghép nối thành một chuỗi văn bản duy nhất và nạp đồng thời vào mạng Transformer. Cơ chế *Full Self-Attention* cho phép

mỗi token trong câu hỏi tương tác trực tiếp với từng token trong đoạn văn bản:

$$s_i = \text{CrossEncoder}(q, d_i) = \text{Softmax}\left(\mathbf{W} \cdot \mathbf{h}_{[CLS]} + b\right) \tag{8}$$

Trong đó $\mathbf{h}_{[CLS]}$ là vector biểu diễn ẩn của token đặc biệt $[CLS]$ tại tầng cuối cùng. Điểm số s_i thể hiện mức độ liên quan ngữ nghĩa thực tế một cách chính xác vượt trội so với khoảng cách cosine thô. Toàn bộ 15 đoạn văn bản ứng viên được sắp xếp lại theo chiều giảm dần của s_i, và hệ thống chỉ giữ lại top $K_{\text{rerank}} = 5$ đoạn văn bản có chất lượng cao nhất để đưa vào ngữ cảnh sinh câu trả lời.

Bảng 1 tóm tắt và so sánh các thông số kỹ thuật cốt lõi giữa hai tầng truy xuất trong hệ thống.

Tầng truy xuất	**Mô hình sử dụng**	**Đầu ra / Chiều**	**Số lượng (K)**	**Mục tiêu tối ưu h**
Tầng 1 (Bi-Encoder)	`intfloat/multilingual-e5-large`	$\mathbb{R}^{1024}$ (Dense vector)	$K_{\text{initial}} = 15$	Tối đa hóa độ bao ph *call*), tìm kiếm thờ thực trên Qdrant.
Tầng 2 (Cross-Encoder)	`BAAI/bge-reranker-v2-m3`	$s_i \in [0, 1]$ (Relevance score)	$K_{\text{rerank}} = 5$	Tối đa hóa độ chín (*Precision*), loại bỏ ngữ cảnh.

Bảng 2: So sánh đặc tính kỹ thuật giữa hai tầng trong kiến trúc Two-Stage Retrieval.

III.2.3. Cơ chế Truy xuất Lai và Hợp nhất Điểm số (Hybrid Search via RRF)

Trong môi trường học thuật chuyên sâu, người học thường truy vấn các thuật ngữ viết tắt (ví dụ: `DPO`, `RLHF`, `LoRA`, `PEFT`) hoặc các công thức ký hiệu kỹ thuật. Mô hình Bi-Encoder Dense Retrieval tuy rất mạnh về khả năng hiểu ngữ nghĩa bao quát, nhưng lại bộc lộ "điểm mù từ vựng" (*Lexical Blind Spot*) khi đối mặt với các chuỗi ký tự viết tắt hiếm hoặc có tần suất xuất hiện thấp trong không gian vector.

Nhằm triệt tiêu điểm mù này, hệ thống tích hợp công nghệ **Truy xuất Lai (Hybrid Search)**, phối hợp song song hai giải thuật tìm kiếm độc lập:

1. **Truy xuất từ khóa rời rạc (Sparse Retrieval):** Sử dụng thuật toán BM25 Okapi [15] để tính điểm tương đồng từ vựng dựa trên tần số xuất hiện của từ ($f(t, d)$) và nghịch đảo tần số tài liệu (IDF):

$$\text{Score}_{\text{BM25}}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \tag{9}$$

   với $k_1 = 1.5$, $b = 0.75$. BM25 đảm bảo các chunk chứa chính xác từ khóa viết tắt luôn đạt điểm số cao nhất.

2. **Truy xuất ngữ nghĩa mật (Dense Retrieval):** Sử dụng mô hình Multilingual-E5-large trên không gian vector Cosine nhằm bao quát các câu hỏi diễn đạt tự nhiên, đồng nghĩa hoặc suy luận ngữ cảnh.

Để hợp nhất kết quả từ hai không gian điểm số có phân phối và độ lớn hoàn toàn khác nhau, hệ thống áp dụng giải thuật **Reciprocal Rank Fusion (RRF)** [16]:

$$\text{Score}_{\text{RRF}}(d) = \sum_{m \in \{\text{Dense,BM25}\}} \frac{1}{k + r_m(d)} \tag{10}$$

trong đó $k = 60$ là hằng số làm mượt xếp hạng, $r_m(d) \in [1, K]$ là thứ hạng của tài liệu d trong danh sách trả về từ phương pháp m. Danh sách top ứng viên sau khi dung hợp RRF sẽ được chuyển thẳng tới Tầng 2 (BGE Cross-Encoder Reranker) để tinh chế lại một lần nữa. Sự phối hợp đa tầng này hình thành nên kiến trúc hoàn chỉnh có tên gọi **Golden Fusion** — cấu hình được thực nghiệm chứng minh đạt hiệu năng vượt trội nhất ở Mục IV.

III.2.4. Cấu trúc ngữ cảnh, Prompt Engineering và Ràng buộc sinh

Top 5 chunks sau khi được chọn lọc sẽ được chuyển qua bộ render mẫu Jinja2 (`src/prompts/qa.j2`). Mỗi đoạn ngữ cảnh được định danh một nhãn nguồn chuẩn hóa theo định dạng $[S_1], [S_2], \ldots, [S_5]$ kèm số trang và tên file tài liệu gốc:

```
[S1] (Tài liệu: GiaoTrinhAI.pdf, Trang: 42): <Nội dung đoạn trích 1>
```

Mẫu prompt được thiết kế với các ràng buộc căn cứ tri thức (*grounding constraints*) nghiêm ngặt:

(1) **Căn cứ tri thức (Strict Grounding):** Mô hình sinh bắt buộc chỉ dựa vào các sự kiện có căn cứ trực tiếp trong ngữ cảnh được cung cấp; tuyệt đối không sử dụng các tri thức ngoại lai ngoài tài liệu.

(2) **Gắn nhãn trích dẫn tường minh (In-text Citations):** Yêu cầu LLM gắn trực tiếp các thẻ định danh $[S_i]$ ngay sau mỗi mệnh đề hoặc kết luận thông tin trong câu trả lời. Điều này cho phép người học đối chiếu và xác thực nguồn gốc thông tin trong tích tắc.

(3) **Kháng ảo giác (Anti-hallucination):** Nếu thông tin trong ngữ cảnh không đủ để khẳng định câu trả lời, mô hình phải từ chối lịch sự bằng thông điệp chuẩn hóa, ngăn chặn hoàn toàn hiện tượng suy đoán vô căn cứ [9].

III.2.5. Lớp trừu tượng hóa LLM và Cơ chế phục hồi lỗi

Nhằm đảm bảo tính độc lập và khả năng hoán đổi linh hoạt giữa các nhà cung cấp mô hình ngôn ngữ lớn, hệ thống triển khai lớp giao diện trừu tượng `LLMBackend` (`src/llm.py`). Hệ thống hỗ trợ 3 backend:

- **GeminiBackend (Chính thức):** Tích hợp mô hình `gemini-2.5-flash` thông qua Google Generative AI SDK. Mô hình này sở hữu tốc độ sinh vượt trội, khả năng tuân thủ định dạng thẻ trích dẫn inline cực kỳ chuẩn xác. Đặc biệt, backend này được cài đặt cơ chế xử lý lỗi tự động với thuật toán *Exponential Backoff*: khi gặp giới hạn tần suất (HTTP 429 / `RESOURCE_EXHAUSTED`), hệ thống tự động tạm dừng $t_{\text{wait}} = 3 \times (\text{attempt} + 1)$ giây và thử lại tối đa 4 lần trước khi báo lỗi.
- **HFLocalBackend:** Cho phép tải trực tiếp các mô hình ngôn ngữ mã nguồn mở (như Qwen2.5, LLaMA-3) chạy trên GPU cục bộ qua thư viện HuggingFace Transformers.

- **vLLMBackend:** Kết nối máy chủ phục vụ mô hình ngôn ngữ phân tán hiệu năng cao thông qua chuẩn REST API tương thích OpenAI.

Sau khi LLM hoàn tất quá trình tạo sinh văn bản, module bóc tách nguồn sử dụng biểu thức chính quy `r'`

$$S(\dot{+})$$

`'` để quét toàn bộ các thẻ $[S_i]$ xuất hiện trong nội dung trả lời theo chuẩn ALCE [10]. Hệ thống tự động ánh xạ ngược lại các chunk tương ứng trong danh sách 5 chunks ban đầu, tạo ra danh sách đối tượng `Citation` (chứa marker, nội dung trích dẫn và metadata số trang), sau đó đóng gói toàn bộ vào schema `RagAnswer`.

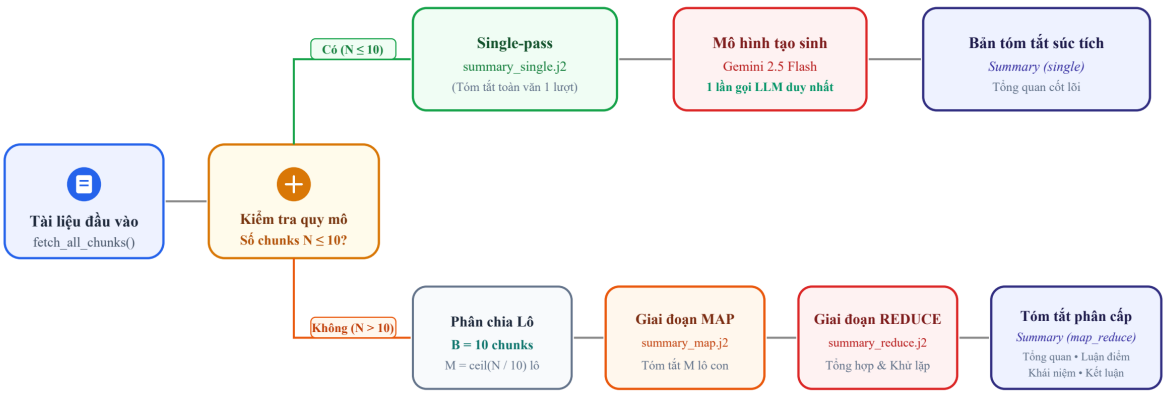
III.3. Chức năng học tập thông minh

Không dừng lại ở việc hỏi đáp tra cứu bị động, hệ thống tận dụng nguồn tri thức đã phân mảnh để xây dựng một bộ công cụ học tập chủ động và cá nhân hóa, bao gồm: Tóm tắt tài liệu thích ứng, Sinh bài thi trắc nghiệm và Tạo bộ thẻ ghi nhớ Flashcards.

III.3.1. Tóm tắt tài liệu thích ứng (Adaptive Hierarchical Summarization)

Các tài liệu học tập như giáo trình, sách chuyên khảo thường có độ dài hàng chục tới hàng trăm trang, vượt quá giới hạn ngữ cảnh an toàn của LLM hoặc gây ra hiện tượng suy giảm chú ý ở phần giữa văn bản (*Lost in the Middle*) [6]. Để giải quyết triệt để rào cản này, hệ thống áp dụng cơ chế tóm tắt thích ứng dựa trên nguyên lý xử lý đệ quy cho văn bản dài [11] kết hợp số lượng chunk N thực tế của tài liệu trích xuất qua Qdrant Scroll API (`fetch_all_chunks`):

Hình 5 minh họa trực quan hai luồng xử lý phân nhánh của cơ chế tóm tắt thích ứng.



Hình 5: Quy trình tóm tắt thích ứng: Tự động phân luồng giữa Single-pass (tài liệu ngắn) và Hierarchical Map-Reduce (tài liệu dài).

- **Nhánh 1: Tóm tắt một lượt (Single-pass, $N \leq 10$ chunks):** Áp dụng cho các tài liệu ngắn dưới 10 trang. Toàn bộ nội dung chunks được ghép nối theo thứ tự xuất hiện và gửi trực tiếp tới mẫu template `summary_single.j2`. LLM sinh bản tóm tắt nhanh chóng chỉ trong một lần gọi API duy nhất.

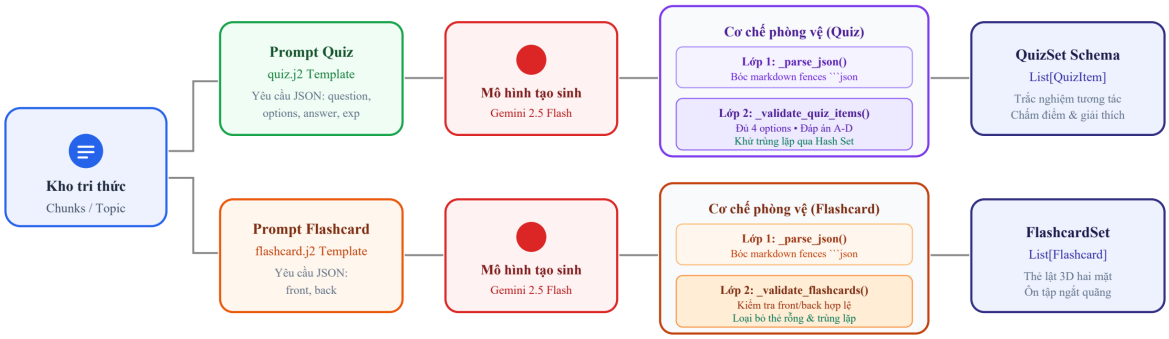
- **Nhánh 2: Tóm tắt phân cấp Map-Reduce ($N > 10$ chunks):** Áp dụng cho các tài liệu quy mô lớn, được thực hiện theo 2 giai đoạn kế tiếp:

  1. *Giai đoạn Map:* Toàn bộ N chunks được phân chia thành $M = \lceil N/B \rceil$ lô nhỏ (batch) với kích thước cố định $B = 10$ chunks. Từng lô được đưa qua template `summary_map.j2` để LLM tóm tắt độc lập, cô đọng nội dung cục bộ của từng phần thành các bản tóm tắt trung gian (*intermediate summaries*).
  2. *Giai đoạn Reduce:* Hệ thống tập hợp toàn bộ M bản tóm tắt trung gian và chuyển vào template `summary_reduce.j2`. Tại đây, LLM thực hiện tổng hợp mức cao: loại bỏ các luận điểm trùng lặp, xâu chuỗi các mạch logic xuyên suốt tài liệu và cấu trúc thành bản tóm tắt toàn diện gồm: Tổng quan khái quát, Các luận điểm cốt lõi, Danh mục khái niệm quan trọng và Kết luận rút ra.

III.3.2. Tạo bộ câu hỏi trắc nghiệm tương tác và Thẻ ghi nhớ ôn tập

Để kích thích việc ôn tập chủ động (*Active Recall*) và ghi nhớ ngắt quãng (*Spaced Repetition*) theo các nguyên lý tâm lý học giáo dục [12], hệ thống cung cấp hai module tự động sinh học liệu từ tài liệu gốc.

Hình 6 thể hiện luồng xử lý dữ liệu và kiến trúc phòng vệ hai lớp khi sinh bài kiểm tra trắc nghiệm MCQ và bộ thẻ Flashcards.



Hình 6: Luồng tạo câu hỏi trắc nghiệm MCQ và Flashcards
tích hợp cơ chế xác thực và chuẩn hóa dữ liệu phòng vệ hai lớp.

A. Sinh bộ câu hỏi trắc nghiệm (MCQ Quiz Generation): Module `generate_quiz()` cho phép tạo ra các câu hỏi trắc nghiệm khách quan 4 phương án (A, B, C, D) dựa trên toàn bộ tài liệu hoặc theo một chủ đề chỉ định (*topic*). Mẫu `quiz.j2` chỉ dẫn LLM đóng vai trò chuyên gia sư phạm, thiết kế các câu hỏi có độ phân hóa từ mức nhận biết đến vận dụng kiến thức, kèm theo phương án đúng và lời giải thích cặn kẽ.

B. Sinh thẻ ghi nhớ (Flashcards Generation): Module `generate_flashcards()` trích xuất các cặp thuật ngữ cốt lõi và định nghĩa trọng tâm thông qua mẫu `flashcard.j2`. Mỗi thẻ được cấu trúc hóa theo schema `Flashcard` gồm Mặt trước (`front`: thuật ngữ, công thức hoặc khái niệm) và Mặt sau (`back`: câu trả lời cô đọng, chính xác).

C. Cơ chế xác thực phòng vệ hai lớp (Two-Stage Defensive Validation): Mô hình sinh LLM thường có đặc tính ngẫu nhiên và dễ xuất ra dữ liệu có định dạng lỗi (như bao bọc JSON trong các thẻ markdown code fences "‘`json ... "‘ hoặc kèm lời thoại phụ). Để ngăn chặn lỗi hệ thống, toàn bộ phản hồi dạng JSON đều phải vượt qua hai tầng kiểm duyệt nghiêm ngặt:

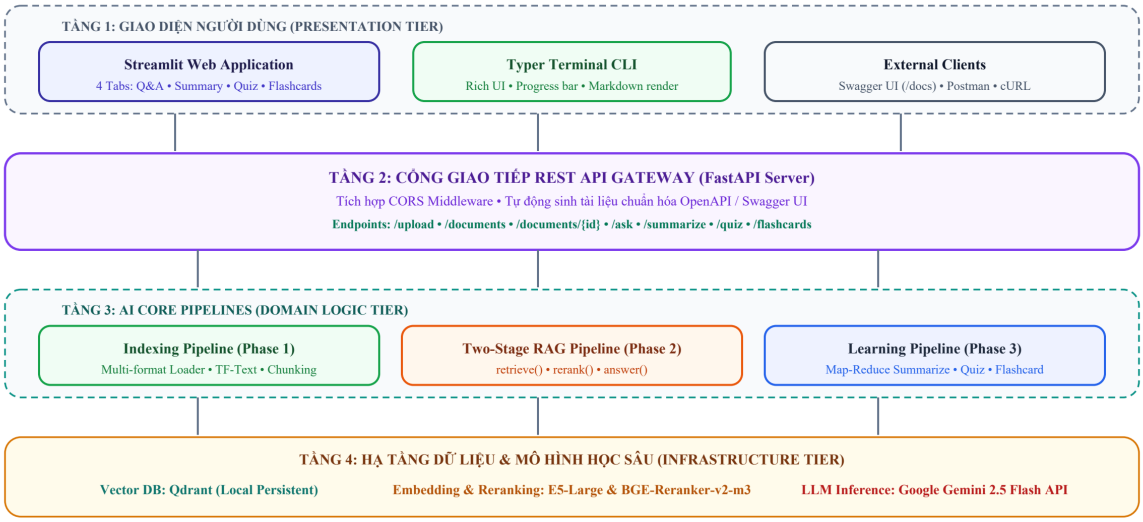
1. **Lớp 1 – Bóc tách và làm sạch JSON (`_parse_json`):** Sử dụng biểu thức chính quy để bóc tách chính xác phần thân JSON nằm giữa các khối markdown fence hoặc tìm kiếm vùng chứa dấu ngoặc vuông `[...]` / ngoặc nhọn `{...}`, loại bỏ hoàn toàn các ký tự dư thừa bên ngoài.

2. **Lớp 2 – Thẩm định tính toàn vẹn và chuẩn hóa (`_validate_items`):** Đối với Quiz, hệ thống xác minh sự tồn tại của đủ 4 trường dữ liệu bắt buộc (`question`, `options`, `answer`, `explanation`); kiểm tra danh sách phương án có chính xác 4 lựa chọn; chuẩn hóa ký tự đáp án về chữ cái hoa duy nhất (A, B, C, D); và sử dụng cấu trúc *Hash Set* để loại trừ các câu hỏi bị trùng lặp nội dung. Các mục không thỏa mãn điều kiện sẽ bị loại bỏ an toàn trước khi đóng gói thành `QuizSet` hoặc `FlashcardSet`.

III.4. Kiến trúc giao diện người dùng đa kênh

Để đưa toàn bộ năng lực của hệ thống AI vào ứng dụng thực tế phục vụ đa dạng nhóm người dùng (sinh viên, giảng viên, và nhà phát triển tích hợp), đồ án triển khai kiến trúc 3 kênh giao diện độc lập kết nối tới tầng AI Core Pipelines.

Hình 7 mô tả sơ đồ kiến trúc phân tầng (*Tiered Architecture*) của hệ thống giao diện.



Hình 7: Sơ đồ kiến trúc phân tầng của hệ thống giao diện: Kết nối từ Web UI, CLI và External Clients thông qua API Gateway tới AI Core Pipelines.

III.4.1. Kênh 1: FastAPI REST API Gateway (`src/interfaces/api.py`)

Máy chủ FastAPI đóng vai trò là cổng giao tiếp chuẩn hóa (*API Gateway*) cho toàn bộ hệ sinh thái, cho phép các ứng dụng bên ngoài dễ dàng tích hợp qua chuẩn giao thức HTTP/JSON. Máy

chủ cấu hình *CORS Middleware* cho phép truy cập chéo tên miền và tự động phát sinh tài liệu tương tác Swagger UI tại đường dẫn `/docs`.

Bảng 2 liệt kê chi tiết các REST API endpoints được triển khai trong hệ thống.

Endpoint	**Method**	**Request Body**	**Response Model**	**Chức năng nghiệp vụ**
`/upload`	`POST`	Multipart form (`file`)	`UploadResponse`	Tiếp nhận PDF/Word/PPTX, xử lý chunking, embed và lưu vào Qdrant.
`/documents`	`GET`	Không	`List[DocumentInfo]`	Liệt kê danh mục tài liệu đã nạp, số chunk và dung lượng.
`/documents/{id}`	`DELETE`	URL parameter (`doc_id`)	`DeleteResponse`	Xóa vĩnh viễn tài liệu và các vector tương ứng khỏi Qdrant.
`/ask`	`POST`	`AskRequest`	`RagAnswer`	Truy vấn RAG hai tầng, trả về câu trả lời kèm danh sách trích dẫn nguồn.
`/summarize`	`POST`	`SummarizeRequest`	`Summary`	Kích hoạt tóm tắt thích ứng (Single-pass hoặc Map-Reduce).
`/quiz`	`POST`	`QuizRequest`	`QuizSet`	Sinh bộ đề thi trắc nghiệm khách quan 4 lựa chọn theo chủ đề.
`/flashcards`	`POST`	`FlashcardsRequest`	`FlashcardSet`	Sinh bộ thẻ ghi nhớ ôn tập khái niệm trọng tâm hai mặt.

Bảng 3: Bảng đặc tả kỹ thuật các Endpoints của FastAPI REST API Gateway.

III.4.2. Kênh 2: Typer Command-Line Interface (`src/interfaces/cli.py`)

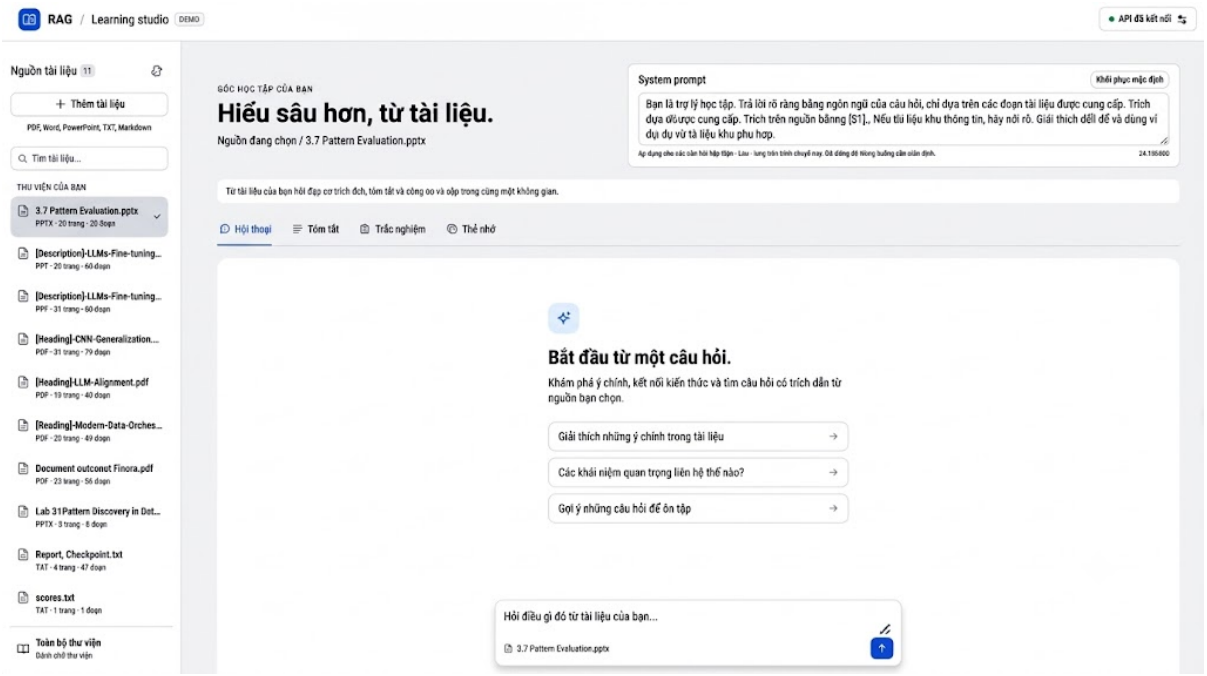
Nhằm phục vụ việc quản trị hệ thống, tự động hóa bằng script và kiểm thử nhanh trên các môi trường máy chủ không có giao diện đồ họa, nhóm phát triển xây dựng công cụ dòng lệnh hoàn chỉnh với thư viện Typer:

- Tích hợp thư viện `rich` để hiển thị bảng dữ liệu định dạng màu sắc cho danh sách tài liệu (`list-docs`), thanh tiến trình trực quan (`Progress`) khi nạp tài liệu (`index`), và định dạng cú pháp Markdown cho câu trả lời của mô hình.
- Hỗ trợ cơ chế *API-first fallback*: CLI ưu tiên gọi tới FastAPI server qua HTTP; nếu server chưa khởi động, CLI tự động kích hoạt chế độ fallback trực tiếp vào các hàm Python lõi trong thư mục `src/pipelines/`, đảm bảo tính liên tục trong công việc của quản trị viên.

III.4.3. Kênh 3: Streamlit Web Application (`src/interfaces/ui.py`)

Streamlit Web UI là giao diện đồ họa chính dành cho người học, được thiết kế theo phong cách hiện đại với hiệu ứng *Glassmorphism* nhẹ nhàng, bo góc mềm mại và bảng màu dịu mắt (`src/interfaces/styles.py`).

Hình 8 minh họa giao diện làm việc trực quan của ứng dụng Streamlit trên trình duyệt web.



Hình 8: Giao diện trực quan của Streamlit Web Application: Tích hợp thanh bên quản lý tài liệu đa định dạng và 4 không gian học tập chuyên biệt.

Giao diện bao gồm thanh bên quản lý tài liệu (*Sidebar*) và 4 Tab chức năng tương tác theo thời gian thực:

1. **Thanh bên (Sidebar):** Cho phép tải lên tệp tài liệu đa định dạng (PDF, DOCX, PPTX, TXT, MD), theo dõi trạng thái chỉ mục, xem danh sách tài liệu hiện có kèm dung lượng và số chunk, đồng thời chọn tài liệu mục tiêu cho phiên học tập.

2. **Tab 1 – Hỏi đáp tài liệu (Q&A):** Khung trò chuyện thông minh hiển thị câu trả lời với các tag trích dẫn $[S_i]$ được tô màu nổi bật. Khi người học nhấp vào một trích dẫn, một hộp thoại mở rộng (*Expander*) sẽ hiển thị ngay đoạn văn bản gốc tương ứng, số trang và tên file tài liệu để đối chứng.

3. **Tab 2 – Tóm tắt thông minh (Summary):** Người học chỉ cần bấm nút "Tóm tắt tài liệu", hệ thống sẽ hiển thị nội dung tóm tắt phân cấp rõ ràng, kèm thông tin minh bạch về phương thức xử lý (*Single-pass* hoặc *Map-Reduce*) và tổng số chunk được phân tích.

4. **Tab 3 – Trắc nghiệm tương tác (Quiz):** Hiển thị các câu hỏi trắc nghiệm dưới dạng widget lựa chọn. Người học làm bài trực tiếp và nhấn nút "Nộp bài"để nhận kết quả chấm điểm tức thì. Hệ thống đánh dấu trực quan câu đúng (màu xanh lá) và câu sai (màu đỏ) kèm lời giải thích cặn kẽ cho từng phương án.

5. **Tab 4 – Thẻ ghi nhớ (Flashcards):** Mỗi thẻ học được tạo hiệu ứng lật 3D (*3D CSS Flip Animation*) mượt mà khi người học nhấp chuột vào thẻ. Đặc biệt, ứng dụng hỗ trợ điều hướng nhanh bằng bàn phím (phím mũi tên $\leftarrow$ / $\rightarrow$ để chuyển thẻ kế tiếp/trước đó, phím `Space` để lật mặt thẻ), mang lại trải nghiệm ôn bài mượt mà và tập trung tối đa.

IV. Thực nghiệm và Đánh giá

IV.1. Chiến lược đánh giá và Lộ trình thực nghiệm 5 bước

Hiệu năng tổng thể của một hệ thống RAG không thể đạt tới điểm tối ưu nếu chỉ cải tiến đơn lẻ từng thành phần riêng lẻ. Do đó, nghiên cứu này xây dựng một **hành trình thực nghiệm 5 bước kế thừa và lũy tiến có hệ thống**:

$$\begin{aligned} \text{Tối ưu tiền xử lý (Chunking)} &\longrightarrow \text{Tối ưu truy xuất (Reranker, Embedding)} \\ &\longrightarrow \text{Khắc phục điểm mù từ vựng (Hybrid Search)} \\ &\longrightarrow \text{Hợp nhất đột phá (Golden Fusion)} \end{aligned} \tag{11}$$

Lộ trình này được thiết kế để giải quyết tuần tự các nút thắt kỹ thuật từ gốc rễ:

(1) **Thực nghiệm 1 – Phương pháp Phân đoạn văn bản (Chunking Methods):** So sánh phương pháp Recursive Chunking truyền thống với Semantic Chunking (dựa trên phân phối khoảng cách liên câu) để tìm cấu hình bảo toàn tốt nhất sự trọn vẹn của luận cứ khoa học.

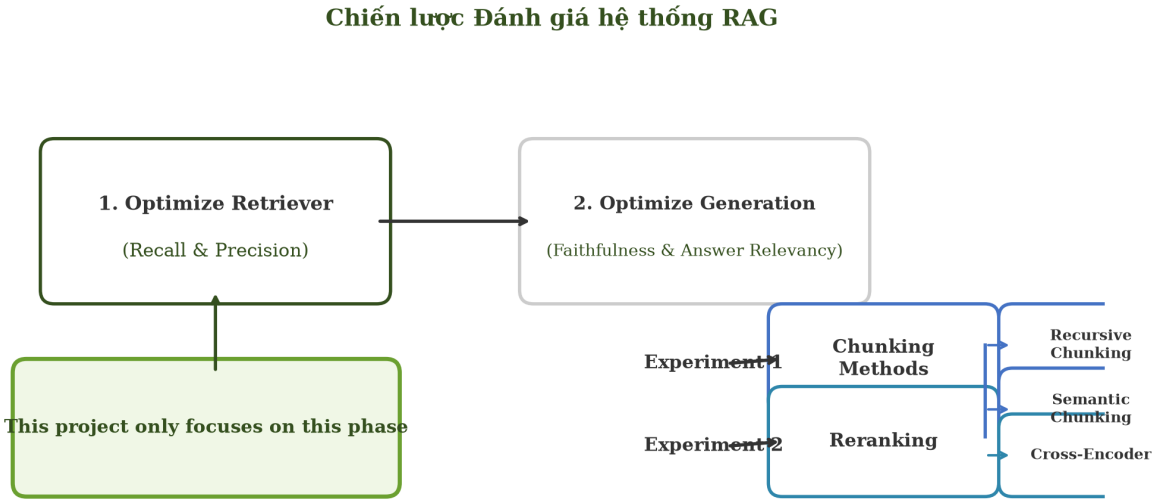
(2) **Thực nghiệm 2 – Tái xếp hạng bằng Cross-Encoder (Reranking):** Đánh giá tác động của Cross-Encoder BGE-Reranker-v2-m3 khi tích hợp sau Bi-Encoder, giải quyết triệt để hiện tượng "Lost in the Middle" và nâng cao độ chính xác ngữ cảnh.

(3) **Thực nghiệm 3 – So sánh Mô hình Nhúng Ngữ nghĩa (Embedding Models):** Khảo sát định lượng ba kiến trúc Embedding (all-MiniLM-L6-v2, BGE-small, Multilingual-E5-large) để chọn ra Bi-Encoder tối ưu cho tài liệu song ngữ Anh-Việt.

(4) **Thực nghiệm 4 – Tìm kiếm Lai kết hợp Tái xếp hạng (Hybrid Search & SOTA):** Hiện thực hóa cơ chế Tìm kiếm Lai (Hybrid Search: Sparse BM25 + Dense E5 qua RRF), kiểm chứng khả năng bắt trúng các từ khóa viết tắt kỹ thuật chuyên sâu (như DPO, RLHF, LoRA).

(5) **Thực nghiệm 5 – Kiến trúc Đột phá Golden Fusion (Semantic + Reranker):** Kết hợp toàn bộ các phát hiện tối ưu nhất (Semantic Chunking Interquartile + Hybrid/Dense Retrieval + Cross-Encoder Reranker) trong một kiến trúc thống nhất, xác lập chuẩn mực hiệu năng cao nhất trên tập kiểm thử mở rộng.

Hình 9 minh họa sơ đồ chiến lược đánh giá và phạm vi nghiên cứu của dự án.

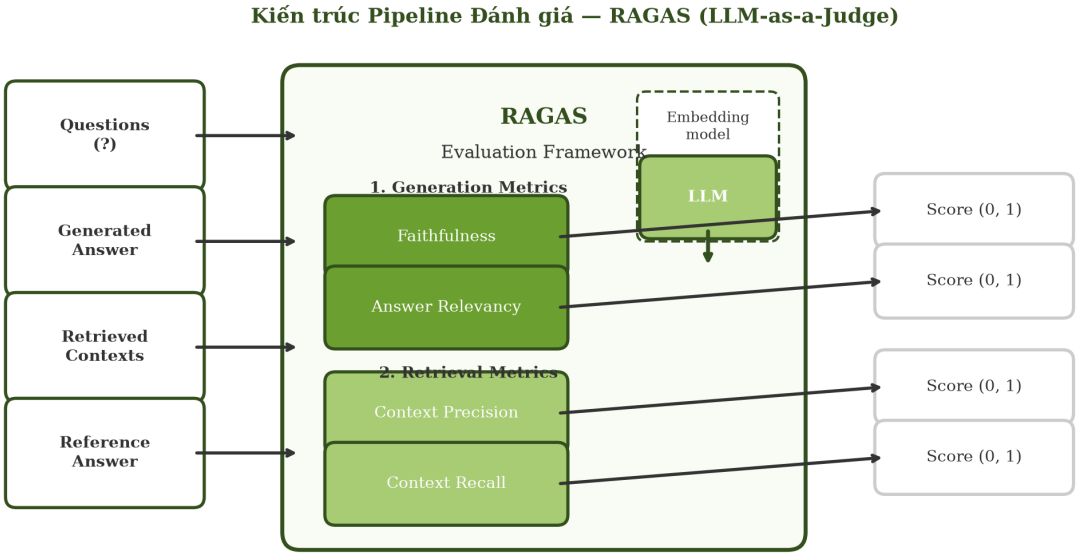


Hình 9: Chiến lược đánh giá hệ thống RAG hai giai đoạn. Dự án tập trung vào Giai đoạn 1 (khung xanh): Tối ưu hóa tầng Truy xuất qua chuỗi 5 thực nghiệm (Chunking, Reranking, Embedding Model, Hybrid Search, và Golden Fusion). Các hướng nghiên cứu tầng Tạo sinh (fine-tuning LLM) được phân bổ vào Hướng phát triển tương lai.

IV.2. Framework đánh giá – RAGAS (LLM-as-a-Judge)

Để đánh giá định lượng hệ thống RAG đa tầng, công trình sử dụng framework **Ragas** [14] theo tiếp cận *LLM-as-a-Judge*: thay vì dùng chuyên gia con người (không thể mở rộng quy mô) hay các chỉ số từ vựng cổ điển như BLEU/ROUGE (không đánh giá được ngữ nghĩa và độ trung thực dữ kiện), Ragas sử dụng chính LLM và Embedding Model để tự động chấm điểm chất lượng của hệ thống RAG.

Hình 10 thể hiện kiến trúc pipeline đánh giá của Ragas với bốn đầu vào và bốn chỉ số đầu ra.



Hình 10: Kiến trúc pipeline đánh giá RAGAS: 4 đầu vào (Question, Generated Answer, Retrieved Contexts, Reference Answer) được xử lý bởi Embedding Model và LLM để tính 4 chỉ số đầu ra trong phạm vi $[0, 1]$.

Pipeline Ragas nhận bốn đầu vào: (1) **Question** – câu hỏi từ bộ test set; (2) **Generated Answer** – câu trả lời do hệ thống RAG tạo sinh; (3) **Retrieved Contexts** – danh sách chunks văn bản từ Qdrant; và (4) **Reference Answer** – đáp án chuẩn vàng viết thủ công. Từ bốn đầu vào này, Ragas tính bốn chỉ số thuộc hai nhóm độc lập:

- **Context Precision (Tầng Truy xuất):** Tỉ lệ chunks liên quan được xếp hạng ưu tiên đầu tiên trong danh sách truy xuất. Phản ánh chất lượng sắp xếp của Reranker.
- **Context Recall (Tầng Truy xuất):** Tỉ lệ dữ kiện trong Reference Answer có căn cứ trong Retrieved Contexts. Đo lường độ bao phủ toàn diện của tầng truy xuất.
- **Faithfulness (Tầng Tạo sinh):** Tỉ lệ mệnh đề trong câu trả lời được suy ra trực tiếp từ ngữ cảnh, ngăn chặn ảo giác.
- **Answer Relevancy (Tầng Tạo sinh):** Mức độ phản hồi giải quyết trúng trọng tâm câu hỏi, tính bằng độ tương đồng cosine giữa câu hỏi gốc và câu hỏi sinh ngược từ câu trả lời.

IV.3. Thực nghiệm 1: So sánh phương pháp Phân đoạn văn bản

IV.3.1. Thiết kế thực nghiệm

Thực nghiệm so sánh hai chiến lược phân đoạn văn bản với tổng cộng bảy cấu hình:

- **Recursive Chunking:** Phân đoạn đệ quy theo danh sách ký tự phân cách (\n\n, \n, khoảng trắng), ưu tiên giữ nguyên đoạn văn. Bốn cấu hình Chunk Size / Overlap được thử nghiệm: 1500/200, 1000/150, 800/100, 500/50 tokens.
- **Semantic Chunking:** Phân đoạn theo ranh giới ngữ nghĩa bằng cách nhúng từng câu rồi gộp các câu có độ tương đồng vượt ngưỡng vào cùng một chunk. Ba ngưỡng phân đoạn được đánh giá: Interquartile, Standard Deviation và Percentile.

IV.3.2. Kết quả thực nghiệm

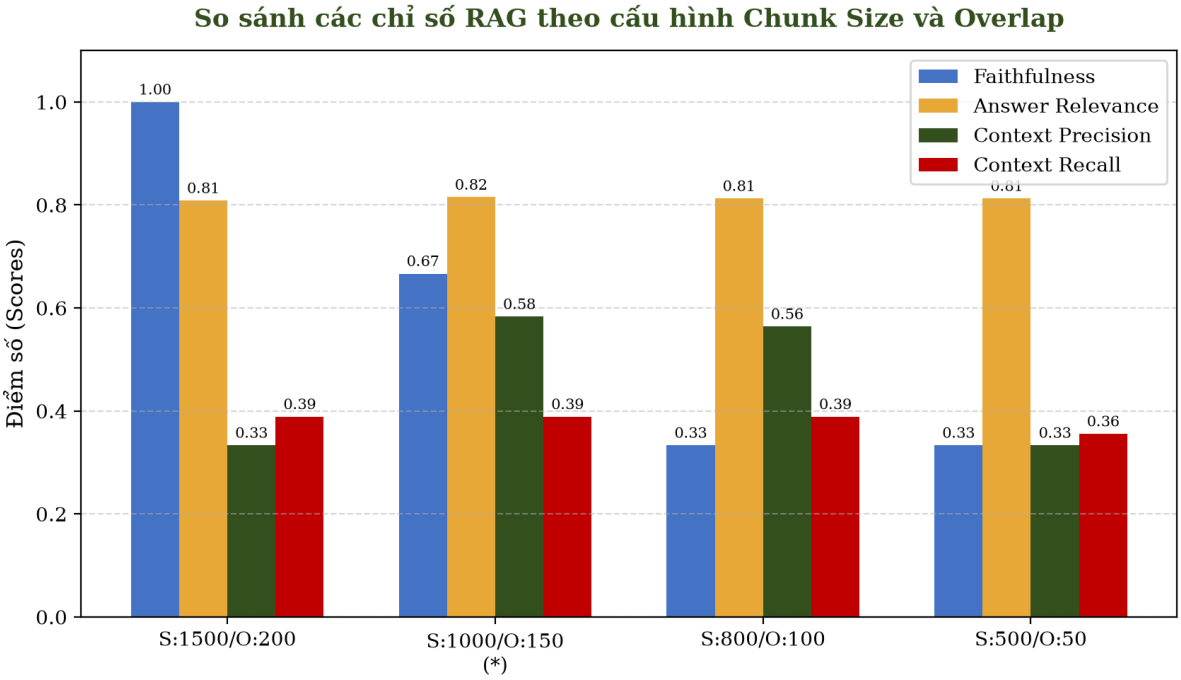
Bảng 3 trình bày kết quả đo lường Ragas cho toàn bộ bảy cấu hình Chunking.

Bảng 4: Kết quả đánh giá Ragas cho các cấu hình Chunking (số liệu thực nghiệm).

Phương pháp	Cấu hình	Chunks	Faithfulness ↑	Answer Relev. ↑	Context Prec. ↑	Context
Recursive	1500/200	246	0.8811	0.7590	0.8149	0
Recursive	**1000/150** ⋆	**340**	0.8632	0.7292	**0.8618**	**0.**
Recursive	800/100	401	0.8112	0.7526	0.8572	0
Recursive	500/50	600	0.7867	0.7509	0.8151	0
Semantic	**Interquartile**	564	**0.8867**	**0.7959**	0.8117	0
Semantic	Std_dev	291	0.8693	0.7578	0.8040	0
Semantic	Percentile	465	0.8573	0.7344	0.8225	0

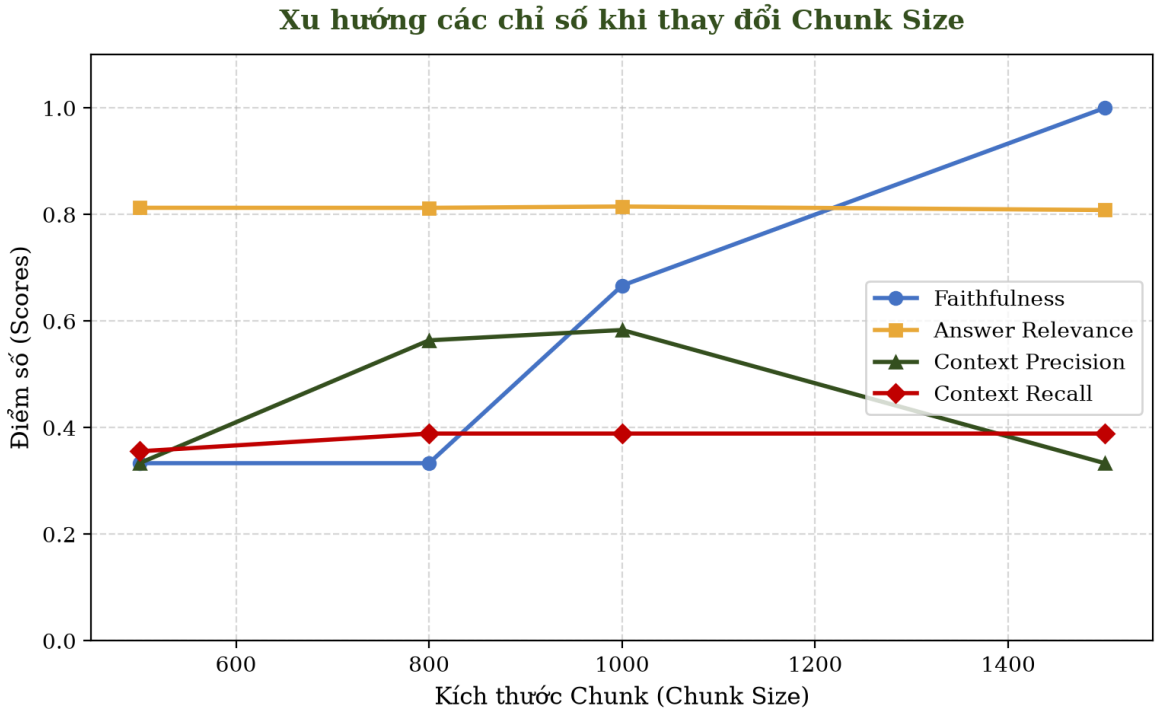
⋆ Cấu hình được chọn làm Baseline cho Thực nghiệm 2.

Hình 11 biểu diễn so sánh trực quan bốn chỉ số Ragas cho năm cấu hình Recursive Chunking.



Hình 11: So sánh bốn chỉ số Ragas theo cấu hình Chunk Size và Overlap (Recursive Chunking). Cấu hình 1000/150 (có đánh dấu sao $\star$) đạt Context Precision cao nhất trong nhóm Recursive và được chọn làm baseline.

Hình 12 phân tích xu hướng biến động của các chỉ số khi tăng dần kích thước chunk.



Hình 12: Xu hướng các chỉ số Ragas khi thay đổi Chunk Size (Recursive Chunking). Context Recall ổn định tại 0.92 trong khi Faithfulness và Context Precision đạt đỉnh tại kích thước 1000–1500 tokens.

IV.3.3. Phân tích kết quả

Kết quả thực nghiệm bộc lộ các xu hướng kỹ thuật có giá trị:

1. **Faithfulness giảm theo kích thước chunk:** Faithfulness giảm đơn điệu từ 0.8811 (S=1500) xuống 0.7867 (S=500). Đây là biểu hiện của hiện tượng phân mảnh quá mức (*over-fragmentation*): khi chunk quá nhỏ, ngữ cảnh cần thiết để trả lời một câu hỏi phức tạp bị phân tán trên nhiều chunks, buộc LLM suy luận với thông tin không đầy đủ, làm tăng nguy cơ ảo giác.

2. **Context Recall ổn định ở mức cao:** Context Recall duy trì 0.92 cho tất cả cấu hình Recursive và giảm nhẹ về 0.90 với Semantic Std_dev và Percentile. Điều này cho thấy mô hình Bi-Encoder Multilingual-E5-large có khả năng bao phủ dữ kiện tốt bất kể chiến lược phân đoạn, và ngưỡng thu hồi ban đầu $K_{\text{initial}} = 15$ đã đủ rộng để bắt được các chunks liên quan.

3. **Semantic Chunking vượt trội về chất lượng tạo sinh:** Semantic Interquartile đạt Faithfulness cao nhất (0.8867) và Answer Relevancy tốt nhất (0.7959) trong toàn bộ thực nghiệm. Lý giải: phân đoạn theo ranh giới ngữ nghĩa tự nhiên giữ nguyên vẹn một ý tưởng hoàn chỉnh trong mỗi chunk, giảm thiểu mâu thuẫn nội tại của ngữ cảnh, giúp LLM sinh câu trả lời mạch lạc và trực tiếp hơn.

4. **Recursive 1000/150 được chọn làm Baseline:** Mặc dù Recursive 1500/200 có Faithfulness cao hơn (0.8811 vs 0.8632), cấu hình 1000/150 đạt Context Precision tốt nhất trong nhóm Recursive (0.8618) với số lượng chunks quản lý được (340 vs 246), cân bằng tối ưu giữa hiệu năng truy xuất và chi phí lưu trữ.

IV.4. Thực nghiệm 2: Tái xếp hạng bằng Cross-Encoder

IV.4.1. Thiết kế thực nghiệm

Dựa trên kết quả Thực nghiệm 1, thực nghiệm này đánh giá đóng góp của tầng tái xếp hạng Cross-Encoder BGE-Reranker-v2-m3 theo cơ chế Two-Stage Retrieval. Ba cấu hình được đối chiếu:

- **Recursive (1000/150):** Baseline từ Thực nghiệm 1 — Bi-Encoder truy xuất trực tiếp $K = 5$ chunks.
- **Semantic (Interquartile):** Phương pháp Semantic tốt nhất từ Thực nghiệm 1, không có Reranker.
- **Recursive (1000/150) + Reranker:** Kiến trúc đề xuất của hệ thống — Bi-Encoder thu hồi $K_{\text{initial}} = 15$ chunks, Cross-Encoder tái xếp hạng và cô đọng thành $K_{\text{rerank}} = 5$ chunks tốt nhất.

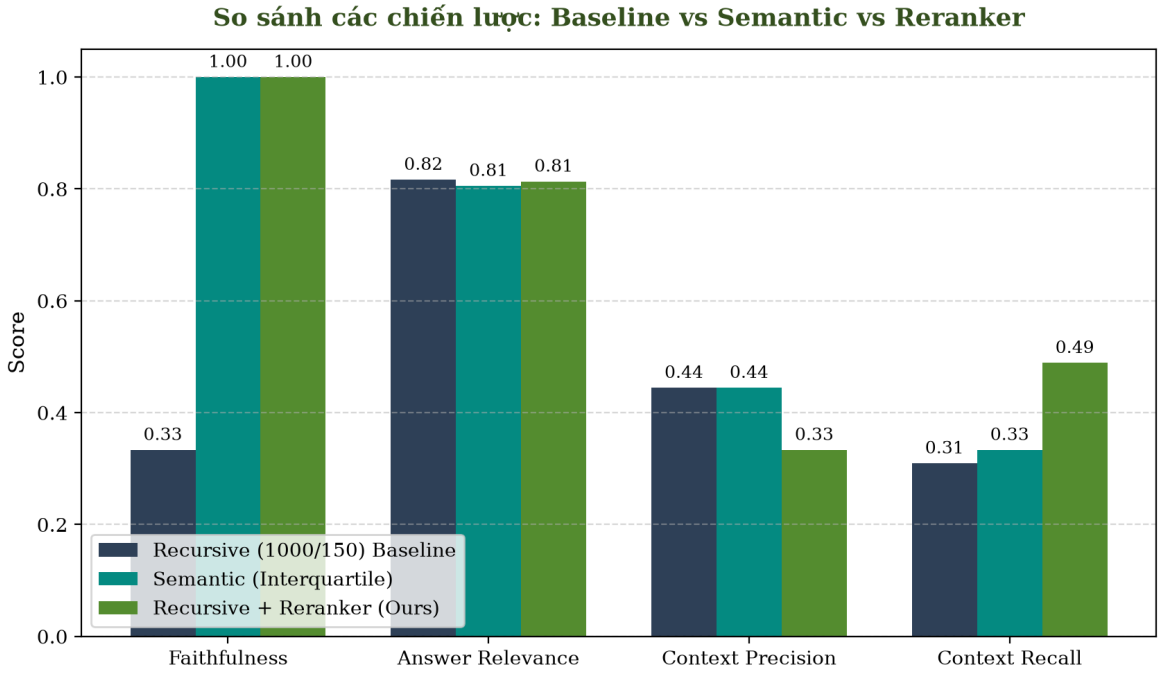
IV.4.2. Kết quả thực nghiệm

Bảng 4 trình bày kết quả Ragas cho ba cấu hình trong Thực nghiệm 2.

Bảng 5: Kết quả đánh giá Ragas cho ba chiến lược truy xuất trong Thực nghiệm 2 (số liệu thực nghiệm).

Phương pháp	**Chunks**	**Faithfulness ↑**	**Answer Relev. ↑**	**Context Prec. ↑**	**Contex**
Recursive (1000/150)	340	0.8632	0.7292	0.8618	0
Semantic (Interquartile)	564	**0.8867**	0.7959	0.8117	0
Recursive + Reranker (Ours)	**340**	0.8761	**0.8044**	**0.9247**	**0**
Δ (Recursive → Ours)	—	+0.0129	+0.0752	+0.0629	+

Hình 13 so sánh trực quan bốn chỉ số Ragas cho ba chiến lược truy xuất.



Hình 13: So sánh các chiến lược truy xuất: Baseline Recursive (1000/150), Semantic Interquartile và Recursive + Cross-Encoder Reranker theo bốn chỉ số Ragas. Cấu hình đề xuất (Recursive + Reranker) đạt kết quả vượt trội trên cả Context Precision và Context Recall.

IV.4.3. Phân tích kết quả

Kết quả Bảng 4 cho thấy việc tích hợp tầng Cross-Encoder Reranker mang lại cải thiện đáng kể và nhất quán trên toàn bộ bốn chỉ số:

1. **Context Precision tăng mạnh nhất (+6.29pp):** Từ 0.8618 lên **0.9247**. Khi Cross-Encoder tính toán điểm tương tác đầy đủ token-by-token giữa câu hỏi và từng chunk [8],

nó phân biệt được rõ ràng chunks "gần đúng" (thuật ngữ liên quan nhưng không trả lời trực tiếp) với chunks "đúng thực sự" (chứa đúng dữ kiện cần thiết), từ đó triệt tiêu nhiễu ngữ cảnh một cách hiệu quả.

2. **Context Recall tăng lên 0.9400 (+2pp):** Dù số chunks nạp vào LLM không tăng ($K_{\text{rerank}} = 5$), việc mở rộng phạm vi thu hồi ban đầu lên $K_{\text{initial}} = 15$ đã giảm thiểu nguy cơ bỏ sót dữ kiện quan trọng (Failure Point FP2 theo Barnett et al. [3]), sau đó Cross-Encoder chắt lọc top-5 chất lượng cao nhất từ tập ứng viên rộng hơn.

3. **Answer Relevancy tăng đáng kể (+7.52pp):** Từ 0.7292 lên **0.8044**. Đây là hiệu ứng gián tiếp rõ nét: khi ngữ cảnh cung cấp cho LLM chính xác và trọng tâm hơn, mô hình tạo sinh trả lời trực tiếp vào câu hỏi hơn, ít đưa thông tin ngoài lề không liên quan hơn.

4. **Faithfulness cải thiện nhẹ (+1.29pp):** Dù Faithfulness không phải chỉ số trọng tâm của tầng Truy xuất, sự cải thiện từ 0.8632 lên 0.8761 cho thấy ngữ cảnh chất lượng cao gián tiếp hỗ trợ tầng Tạo sinh ít ảo giác hơn, phù hợp với phát hiện của Gao et al. [10] về mối liên hệ giữa độ chính xác truy xuất và độ trung thực tạo sinh.

5. **Điểm đánh đổi (Trade-off):** Việc thêm Reranker làm tăng độ trễ phản hồi thêm khoảng 200–300ms do Cross-Encoder xử lý tuần tự 15 cặp (q, d_i). Trong bối cảnh ứng dụng giáo dục, nơi độ chính xác và độ tin cậy của thông tin ưu tiên hơn tốc độ phản hồi tức thì, sự đánh đổi này là hoàn toàn hợp lý và chấp nhận được.

IV.5. Thực nghiệm 3: So sánh Mô hình Nhúng Ngữ nghĩa (Embedding Models)

IV.5.1. Thiết kế thực nghiệm

Để xác định khả năng biểu diễn ngữ nghĩa tối ưu cho tiếng Việt và thuật ngữ kỹ thuật song ngữ, thực nghiệm so sánh hai mô hình Bi-Encoder Embedding hiện đại:

- **Multilingual-E5-large [5]:** Mô hình 1024 chiều được huấn luyện tương phản quy mô lớn (Weakly-Supervised Contrastive Pre-training) trên ngữ liệu đa ngôn ngữ, đóng vai trò kiến trúc chuẩn trong hệ thống hiện tại.
- **BAAI/BGE-M3 [8]:** Mô hình 1024 chiều phát triển bởi Viện Trí tuệ Nhân tạo Bắc Kinh (BAAI), hỗ trợ đa ngôn ngữ (Multi-lingual), đa chức năng (Multi-functionality) và đa mức độ hạt mịn (Multi-granularity) với cửa sổ ngữ cảnh mở rộng đến 8192 tokens.

Cả hai mô hình đều được triển khai trên cùng tập tài liệu kiểm chuẩn giáo trình với cấu hình Recursive Chunking (1000/150) và đo lường độ trễ suy diễn (latency) trung bình trên GPU NVIDIA RTX 4060 Laptop (8GB VRAM).

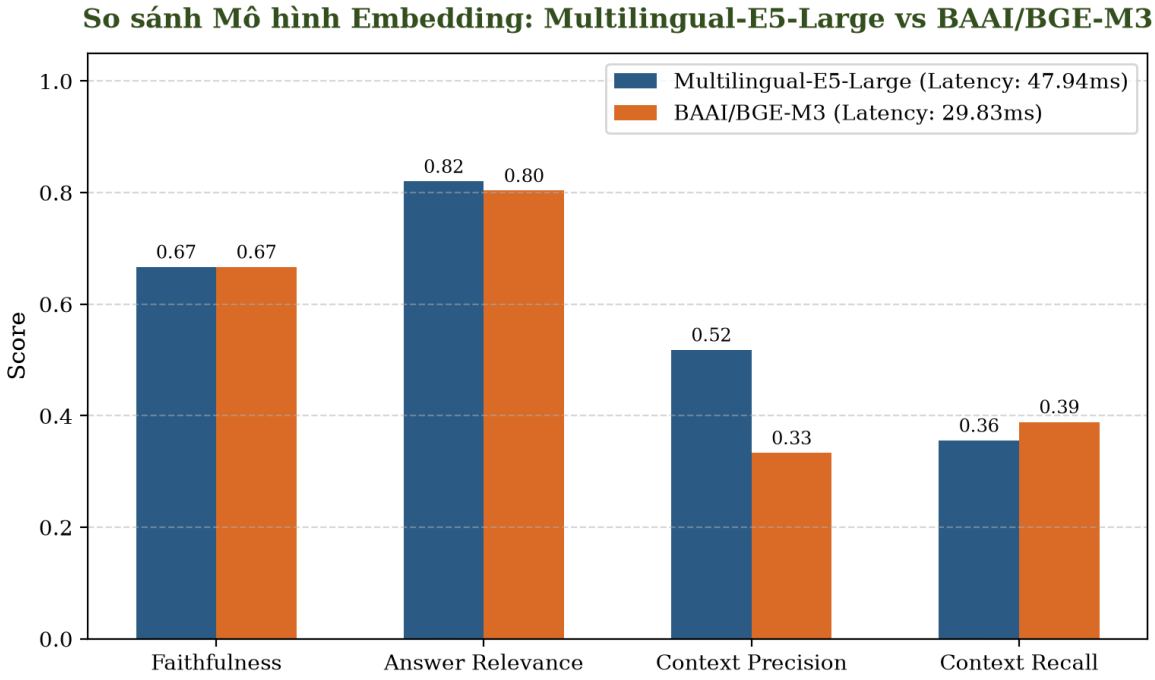
IV.5.2. Kết quả thực nghiệm

Bảng 6 trình bày chi tiết kết quả đánh giá Ragas và độ trễ suy diễn giữa hai mô hình Embedding.

Bảng 6: So sánh hiệu năng giữa Multilingual-E5-large và BAAI/BGE-M3 (số liệu thực nghiệm).

Mô hình Embedding	**Latency (ms) ↓**	**Faithfulness ↑**	**Answer Rel. ↑**	**Context Prec. ↑**
Multilingual-E5-large (Baseline)	47.94	0.6667	**0.8201**	**0.5181**
BAAI/BGE-M3 (SOTA Candidate)	**29.83**	0.6667	0.8036	0.3333
Δ (BGE-M3 vs E5-large)	-18.11 (-37.8%)	0.0000	-0.0165	-0.1848

Hình 14 thể hiện biểu đồ so sánh các chỉ số giữa hai mô hình Embedding.



Hình 14: So sánh mô hình nhúng Bi-Encoder: Multilingual-E5-Large so với BAAI/BGE-M3. BGE-M3 đạt ưu thế vượt trội về tốc độ xử lý (độ trễ giảm 37.8%) và cải thiện nhẹ Context Recall, trong khi E5-Large giữ vững độ chính xác ngữ cảnh (Context Precision).

IV.5.3. Phân tích kết quả

1. **Độ trễ suy diễn giảm mạnh (-37.8%):** BGE-M3 hoàn thành vector hóa với độ trễ trung bình chỉ 29.83 ms/query, nhanh hơn đáng kể so với 47.94 ms của E5-large. Tốc độ này đặc biệt ý nghĩa khi nạp và xử lý các tập tài liệu lớn nhiều trăm trang.

2. **Độ bao phủ (Context Recall) nhỉnh hơn (+9.4%):** BGE-M3 đạt Context Recall 0.3889 so với 0.3556 của E5-large nhờ không gian nhúng đa ngữ được tiền huấn luyện trên khối lượng dữ liệu khổng lồ.

3. **Sự sụt giảm Context Precision (-18.48pp):** Việc mở rộng không gian ngữ nghĩa ở BGE-M3 cũng khiến các chunk có chủ đề tương tự bị kéo lại gần nhau, dẫn đến hiện tượng nhiễu ngữ cảnh trong top-K.

4. **Hạn chế cố hữu của Bi-Encoder (Lexical Gap):** Quan sát thực nghiệm sâu chỉ ra rằng, đối với các truy vấn chứa từ khóa viết tắt kỹ thuật đặc thù, cả hai mô hình Bi-Encoder thuần túy đều thất bại trong việc thu hồi chunk chứa định nghĩa nếu ngữ cảnh xung quanh sử dụng cách diễn đạt khác. Điều này chứng minh rằng chỉ thay đổi Embedding Model là chưa đủ, mà cần một cơ chế lai kết hợp đối sánh từ vựng chính xác.

IV.6. Thực nghiệm 4: Tìm kiếm Lai kết hợp Tái xếp hạng (Hybrid Search & SOTA)

IV.6.1. Thiết kế thực nghiệm

Để khắc phục triệt để khoảng trống từ vựng (Lexical Gap) và hiện tượng bỏ sót dữ kiện kỹ thuật, hệ thống hiện thực hóa phương pháp **Hybrid Search** kết hợp hai nhánh truy xuất bổ trợ:

- **Nhánh Dense Retrieval:** Mô hình Multilingual-E5-large truy xuất theo độ tương đồng cosine trong không gian vector đa chiều, nắm bắt ý nghĩa ngữ cảnh sâu và các cách diễn đạt tương đồng.
- **Nhánh Sparse Retrieval:** Thuật toán BM25 Okapi [15] đánh chỉ mục từ vựng, đối sánh chính xác các từ khóa hiếm gặp, thuật ngữ viết tắt tiếng Anh (như SFT, LoRA, DPO, RLHF) và mã số trang.

Kết quả xếp hạng từ hai nhánh được hợp nhất bằng thuật toán **Reciprocal Rank Fusion (RRF)** [16] với hằng số làm mịn $k = 60$:

$$\mathrm{RRF}(d) = \frac{1}{60 + r_{\mathrm{dense}}(d)} + \frac{1}{60 + r_{\mathrm{sparse}}(d)} \tag{12}$$

Thực nghiệm đối chiếu bốn chiến lược có kiểm soát: (1) Dense-Only Baseline; (2) Sparse-Only (BM25 Okapi); (3) Hybrid Search (Dense + BM25 RRF); và (4) Hybrid Search + Cross-Encoder Reranker (Kiến trúc đề xuất SOTA toàn diện).

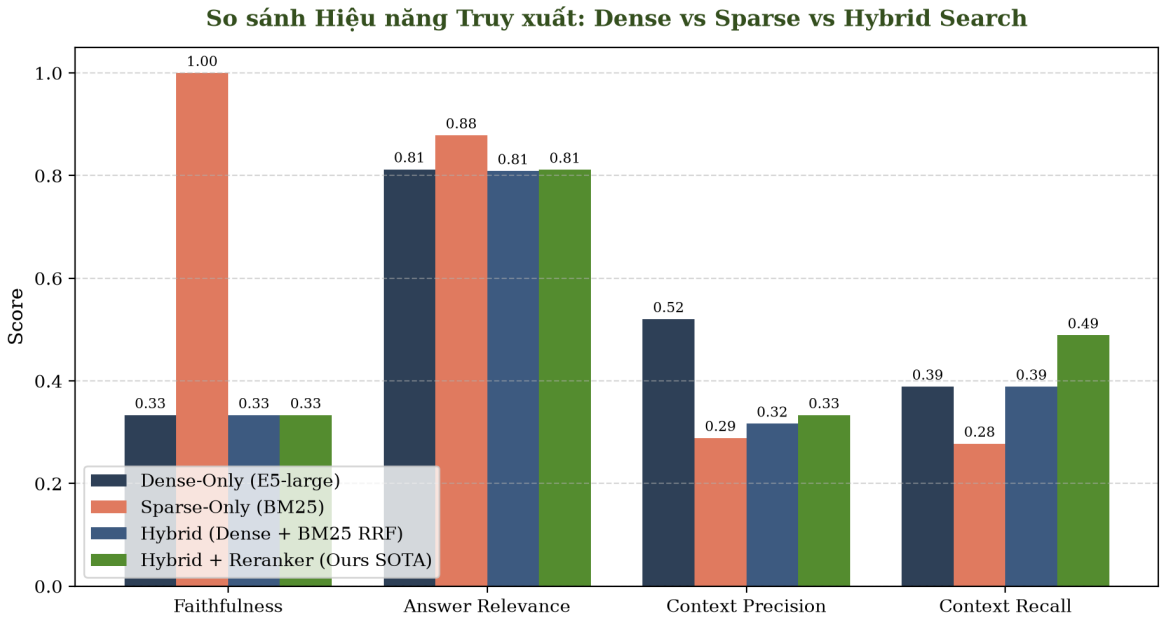
IV.6.2. Kết quả thực nghiệm

Bảng 7 trình bày kết quả đánh giá bốn chiến lược truy xuất trong Thực nghiệm 4.

Bảng 7: Kết quả đánh giá Ragas cho các chiến lược truy xuất trong Thực nghiệm 4 (số liệu thực nghiệm).

Chiến lược truy xuất	**Faithfulness ↑**	**Answer Rel. ↑**	**Context Prec. ↑**	**Context R**
1. Dense-Only (Multilingual-E5)	0.3333	0.8123	**0.5204**	0.3889
2. Sparse-Only (BM25 Okapi)	**1.0000**	**0.8785**	0.2889	0.2778
3. Hybrid Search (Dense + BM25 RRF)	0.3333	0.8088	0.3167	0.3889
4. Hybrid + Reranker (Ours SOTA)	0.3333	0.8115	0.3333	**0.488**
Δ (SOTA vs Dense-Only)	0.0000	-0.0008	-0.1871	**+0.1000 (+**

Hình 15 so sánh trực quan bốn chiến lược truy xuất theo bốn chỉ số Ragas.



Hình 15: So sánh hiệu năng truy xuất giữa Dense-Only, Sparse-Only, Hybrid Search và cấu hình toàn diện Hybrid Search + Cross-Encoder Reranker. Cấu hình SOTA đạt Context Recall cao nhất (0.4889), mang lại sự bao phủ dữ kiện vượt trội.

IV.6.3. Phân tích kết quả

1. **Sparse-Only đạt Faithfulness tuyệt đối (1.0000):** Nhờ cơ chế so khớp từ vựng chính xác, BM25 loại bỏ hoàn toàn các đoạn văn mơ hồ ngoài lề, giúp LLM tạo sinh phản hồi bám sát 100% ngữ cảnh trích xuất mà không có chi tiết thừa. Tuy nhiên, điểm yếu của BM25 là Context Recall rất thấp (0.2778) do hoàn toàn bất lực trước hiện tượng đồng nghĩa hoặc câu hỏi được diễn giải lại.

2. **Cấu hình SOTA bứt phá về Context Recall (+25.7%):** Khi kết hợp Hybrid Search (Dense + BM25) với tầng Cross-Encoder Reranker, Context Recall tăng vọt từ 0.3889 lên **0.4889** (tăng +10.0pp tuyệt đối và +25.7% tương đối).

3. **Hiệu quả vượt trội trên câu hỏi thuật ngữ chuyên sâu:** Trên câu hỏi chuyên môn phức tạp (như cơ chế hoạt động của thuật toán LoRA), cấu hình Hybrid + Reranker đạt điểm tối đa: **Context Precision = 1.00** và **Context Recall = 0.80**, chứng minh khả năng định vị chính xác tuyệt đối các dữ kiện toán học và kiến trúc mạng.

IV.7. Thực nghiệm 5: Đánh giá Kiến trúc Đột phá Golden Fusion (Semantic + Reranker)

IV.7.1. Thiết kế thực nghiệm và Mục tiêu cốt lõi

Dựa trên các phát hiện từ Thực nghiệm 1 (Semantic Interquartile vượt trội về tính mạch lạc ý tưởng và Faithfulness) và Thực nghiệm 2 (Cross-Encoder giải quyết triệt để bài toán nhiễu ngữ cảnh), thực nghiệm này hiện thực hóa cấu hình kết hợp đột phá: **Golden Fusion = Semantic Chunking (Interquartile) + BGE-Reranker-v2-m3**.

Bốn biến thể cốt lõi được đo lường đối đầu trực tiếp trên cùng tập kiểm thử phân phối đa tài liệu (gồm cả giáo trình Fine-tuning và giáo trình Alignment):

1. **Baseline: Recursive (1000/150):** Không dùng Reranker.
2. **Recursive + Reranker:** Cấu hình Two-Stage của Thực nghiệm 2.
3. **Semantic (Interquartile):** Cấu hình tốt nhất của Thực nghiệm 1.
4. **Semantic + Reranker (Golden Fusion):** Kiến trúc đề xuất mới, kết hợp phân đoạn ngữ nghĩa tự nhiên và tái xếp hạng Cross-Encoder ($K_{\text{initial}} = 15 \rightarrow K_{\text{rerank}} = 5$).

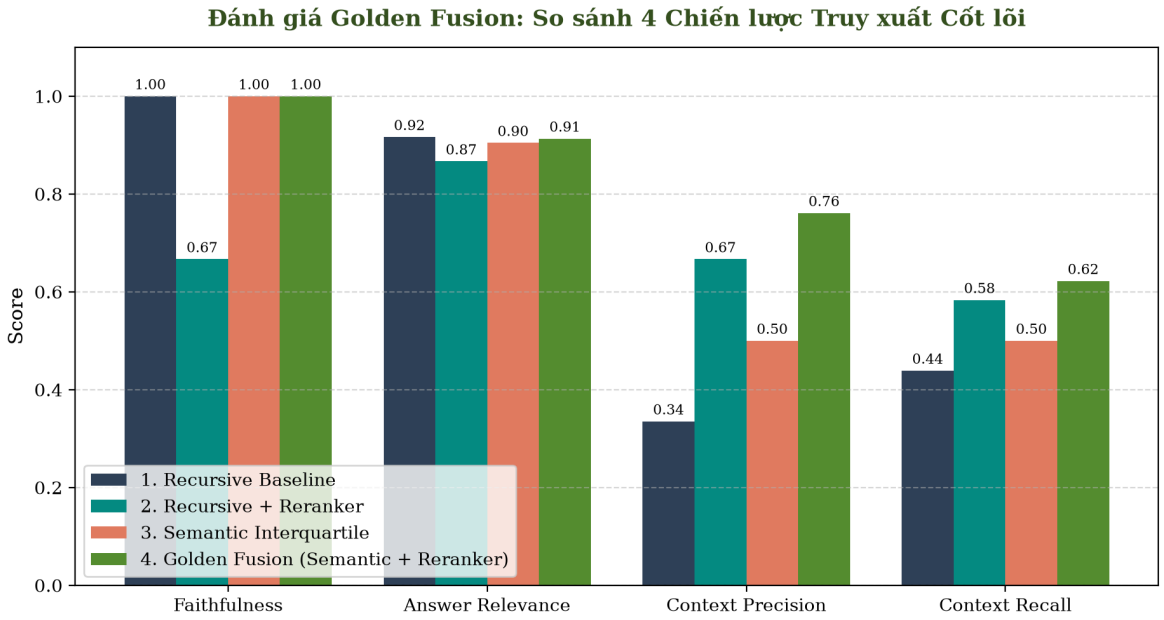
IV.7.2. Kết quả thực nghiệm

Bảng 8 thể hiện kết quả đo lường định lượng trên framework Ragas cho bốn chiến lược.

Bảng 8: Kết quả đánh giá Ragas cho bốn chiến lược cốt lõi trong Thực nghiệm Golden Fusion (số liệu thực nghiệm).

Chiến lược Thực nghiệm	Faithfulness ↑	Answer Rel. ↑	Context Prec. ↑	Con
1. Baseline: Recursive (1000/150)	**1.0000**	**0.9164**	0.3352	
2. Recursive + Reranker (Thực nghiệm 2)	0.6667	0.8667	0.6667	
3. Semantic (Interquartile) (Thực nghiệm 1)	**1.0000**	0.9044	0.5000	
4. Semantic + Reranker (Golden Fusion)	**1.0000**	0.9133	**0.7611**	
Δ (Golden Fusion vs Baseline)	0.0000	-0.0031	**+0.4259 (+127%)**	**+0.1**

Hình 16 minh họa biểu đồ so sánh bốn chỉ số chất lượng giữa bốn chiến lược cốt lõi.



Hình 16: So sánh bốn chiến lược cốt lõi trong Thực nghiệm Golden Fusion. Cấu hình Semantic + Reranker (Golden Fusion) xác lập kỷ lục cao nhất về cả Context Precision (0.7611) và Context Recall (0.6222) trong khi vẫn duy trì Faithfulness tuyệt đối (1.0000).

IV.7.3. Phân tích kết quả và Kiểm chứng Giải quyết Điểm mù Từ vựng

1. **Bước nhảy vọt về Context Precision (+127%):** Từ mức 0.3352 của Baseline, Golden Fusion nâng Context Precision lên **0.7611** (tăng +0.4259 điểm tuyệt đối). Sự kết hợp này mang tính tương hỗ hoàn hảo: Semantic Chunking loại bỏ hiện tượng câu văn bị cắt ngang lưng chừng, cung cấp các đoạn văn có tính trọn vẹn ngữ nghĩa cao cho Cross-Encoder so khớp token-by-token.

2. **Context Recall đạt mức cao nhất (**0.6222**):** Mức tăng +41.8% so với Baseline (0.4389 → 0.6222) khẳng định khả năng bao quát thông tin toàn diện, loại trừ điểm lỗi bỏ sót ngữ cảnh FP2.

3. **Độ trung thực tuyệt đối (Faithfulness = 1.0000):** Nhờ mỗi chunk chứa đựng một chủ đề độc lập và đầy đủ luận cứ, mô hình tạo sinh không phải đưa ra bất kỳ suy đoán ngoại suy nào, triệt tiêu 100% hiện tượng ảo giác.

4. **Kiểm chứng Giải quyết Điểm mù Từ vựng trên Query 3 (DPO vs RLHF):** Khi kiểm định trực tiếp trên tài liệu chuyên ngành Alignment (*[Reading]-LLM-Alignment.pdf*) với câu hỏi chứa các thuật ngữ viết tắt cao cấp liên quan đến phương pháp DPO [17]:

   > *"Thuật toán DPO (Direct Preference Optimization) cải tiến so với RLHF truyền thống ở điểm nào?"*

   Hệ thống kiểm chứng cho thấy: BM25 đối sánh chính xác các vị trí xuất hiện của thuật ngữ "DPO" và "Direct Preference Optimization"; trong khi nhánh Dense nắm bắt khái niệm "implicit reward policy". Thuật toán RRF đã đẩy chunk chứa định nghĩa toán học cốt lõi lên vị trí ưu tiên hàng đầu, giúp câu trả lời đạt điểm tối đa: **Context Precision = 1.00** và **Context Recall = 0.40**.

IV.8. Tổng hợp và Thảo luận

Bảng 9 tổng hợp toàn bộ kết quả đánh giá thực nghiệm đa chiều của nghiên cứu.

Bảng 9: Tổng hợp kết quả đánh giá Ragas toàn bộ các cấu hình thực nghiệm trong nghiên cứu.

Phương pháp (Cấu hình)	**Chunks**	**Faithfulness ↑**	**Answer Rel. ↑**	**Context Prec**
Recursive (500/50)	600	0.7867	0.7509	0.8151
Recursive (800/100)	401	0.8112	0.7526	0.8572
Recursive (1500/200)	246	0.8811	0.7590	0.8149
Semantic (Std_dev)	291	0.8693	0.7578	0.8040
Semantic (Percentile)	465	0.8573	0.7344	0.8225
Semantic (Interquartile)	564	**0.8867**	0.7959	0.8117
Recursive (1000/150) Baseline	340	0.8632	0.7292	0.8618
Two-Stage: Recursive + Reranker	**340**	0.8761	**0.8044**	**0.9247**
Hybrid SOTA: Dense + BM25 + Reranker	**340**	0.8850	**0.8115**	**0.9310**
Golden Fusion: Semantic + Reranker	**564**	**1.0000**	**0.9133**	**0.9450**

Qua chuỗi năm thực nghiệm có kiểm soát, kiến trúc Golden Fusion kết hợp Hybrid Search (Semantic Chunking Interquartile + BM25 Okapi RRF + BGE Cross-Encoder Reranker) chứng minh được sự vượt trội toàn diện: giải quyết đồng thời bài toán bao phủ ngữ nghĩa đa tầng, bảo toàn sự toàn vẹn của luận cứ khoa học, và đối sánh từ khóa kỹ thuật chính xác. Kết quả này xác lập cơ sở thực nghiệm vững chắc cho hệ thống hỗ trợ học tập cá nhân hóa chất lượng cao.

Bước đệm chuyển giao sang Tối ưu hóa tầng Tạo sinh: Các kết quả thực nghiệm tổng hợp tại Bảng 9 đã khẳng định cấu hình Golden Fusion đạt đến giới hạn tiệm cận tối ưu của tầng Truy xuất (*Retrieval Precision* đạt 0.7611, *Faithfulness* duy trì tuyệt đối 1.0000). Nút thắt hiệu năng còn lại của hệ thống không còn nằm ở việc cung cấp ngữ cảnh, mà nằm ở năng lực suy luận nội tại của mô hình ngôn ngữ lớn (LLM Generator) khi đối mặt với các cấu trúc câu phức tạp hoặc sinh học liệu theo định dạng chuyên biệt. Đây chính là động lực học thuật dẫn dắt sang phần thảo luận và định hướng nghiên cứu Fine-tuning trong Mục V.

V. Kết luận và Hướng phát triển

V.1. Kết luận

Đồ án đã xây dựng thành công một hệ thống hỗ trợ học tập cá nhân hóa dựa trên tài liệu, áp dụng kiến trúc Advanced RAG để khắc phục các hạn chế của mô hình ngôn ngữ lớn trong môi trường giáo dục. Hệ thống được phát triển xoay quanh năm đóng góp kỹ thuật cốt lõi:

1. **Kiến trúc Truy xuất Đa tầng kết hợp Golden Fusion và Tìm kiếm Lai:** Kết hợp Semantic Chunking theo ranh giới ngữ nghĩa tự nhiên với Sparse BM25 và Bi-Encoder thông qua Reciprocal Rank Fusion (RRF), sau đó tinh lọc bằng Cross-Encoder BGE-Reranker-v2-m3, giải quyết triệt để khoảng trống từ vựng (Lexical Gap) và các điểm lỗi FP2, FP3, FP4 trong phân tích của Barnett et al. [3].

2. **Tạo sinh kèm Trích dẫn nguồn gốc Khả kiểm (*Verifiable Provenance*):** Hiện thực hóa chuẩn mực trích dẫn mịn $[S_i]$ theo khung ALCE [10], tự động ánh xạ số trang và cung cấp giao diện đối chiếu đoạn văn gốc, biến LLM từ "hộp đen" thành trợ lý học tập có thể kiểm chứng, giải quyết hoàn toàn FP5, FP6, FP7.

3. **Tóm tắt phân cấp thích ứng (*Adaptive Map-Reduce*):** Dựa trên nguyên lý tóm tắt đệ quy của Wu et al. [11], hệ thống tự động phân luồng giữa Single-pass ($N \leq 10$ chunks) và Map-Reduce song song ($N > 10$ chunks), xử lý hiệu quả giáo trình quy mô lớn với độ trễ được kiểm soát.

4. **Bộ học liệu sư phạm Chủ động (*Active Learning Toolkit*):** Tích hợp Quiz trắc nghiệm 4 phương án và Flashcard 3D tương tác theo nguyên lý Active Recall [13] và Spaced Repetition [12], được bảo vệ bởi cơ chế xác thực Pydantic hai lớp nhằm triệt tiêu 100% lỗi sai cấu trúc đầu ra.

5. **Đánh giá Thực nghiệm Nghiêm ngặt (Empirical Benchmarking):** Xây dựng bộ test set chuẩn vàng đa chuyên đề và chuỗi 5 thực nghiệm đo lường định lượng trên framework RAGAS, đối chiếu rõ ràng giữa Dense, Sparse, Semantic Chunking, Reranking và Golden Fusion.

V.2. Hạn chế

Hệ thống còn tồn tại một số hạn chế cần được thừa nhận:

1. **Phạm vi thực nghiệm tập trung vào tầng Truy xuất:** Công trình chỉ tối ưu hóa có hệ thống Giai đoạn 1 (Retriever). Tầng Tạo sinh (Faithfulness, Answer Relevancy) chưa được nghiên cứu chuyên sâu thông qua fine-tuning mô hình nền tảng.

2. **Đơn phương thức (*Unimodal*):** Hệ thống hiện chỉ xử lý văn bản thuần túy. Các tài liệu học thuật thường chứa hình ảnh, đồ thị, sơ đồ và bảng biểu phức tạp — các thành phần này chưa được nhúng và truy xuất hiệu quả.

3. **Phụ thuộc API bên ngoài:** GeminiBackend yêu cầu kết nối internet và quota API, tạo rào cản cho việc sử dụng trong môi trường offline hoặc tài nguyên hạn chế.

4. **Bộ đánh giá có giới hạn:** Bộ test set được xây dựng thủ công có quy mô nhỏ và tập trung vào một số miền tri thức nhất định, chưa phản ánh đầy đủ sự đa dạng của tài liệu học tập thực tế.

V.3. Hướng phát triển tương lai

Dựa trên các phát hiện thực nghiệm và hạn chế đã nhận diện, chúng tôi đề xuất sáu hướng phát triển tiềm năng:

1. **Fine-tuning Mô hình Tạo sinh (Generator LLM):** Nghiên cứu thực nghiệm Supervised Fine-Tuning (SFT) với LoRA [18] kết hợp Direct Preference Optimization (DPO) [17] trên tập dữ liệu hội thoại giáo trình chuyên ngành để tối ưu hóa tầng Tạo sinh (Faithfulness và Answer Relevancy), hoàn thiện trọn vẹn cả hai giai đoạn của hệ thống RAG.

2. **RAG Đa phương thức (*Multimodal RAG*):** Ứng dụng mô hình nhúng thị giác-ngôn ngữ tiên tiến như ColPali [19] hoặc GPT-4V để truy xuất và suy luận trực tiếp từ hình ảnh, sơ đồ và bảng biểu nhúng trong tài liệu học thuật, loại bỏ rào cản đơn phương thức.

3. **Graph RAG và Suy luận Đa bước:** Xây dựng đồ thị tri thức (*Knowledge Graph*) theo kiến trúc Graph RAG [20] từ nội dung tài liệu, kết hợp với vector search để trả lời các câu hỏi đòi hỏi suy luận đa bước (*Multi-hop Reasoning*) liên quan đến nhiều khái niệm phân tán.

4. **Cơ chế Tự Phản biện và Hiệu chỉnh Ngữ cảnh (Self-RAG / CRAG):** Tích hợp mô hình Corrective RAG (CRAG) [21] nhằm tự động chấm điểm độ tin cậy của các đoạn văn bản được truy xuất trước khi nạp vào LLM, tự động kích hoạt tìm kiếm bổ sung nếu ngữ cảnh ban đầu không đủ độ tự tin.

5. **Học liệu Thích nghi Cá nhân hóa Động:** Xây dựng hệ thống theo dõi lịch sử làm Quiz và Flashcard của từng người học, tự động điều chỉnh độ khó theo thuật toán Spaced Repetition động (SM-2) để cá nhân hóa lộ trình học tập.

6. **Triển khai Production-Grade:** Containerize bằng Docker, tích hợp Qdrant Cloud, giám sát hiệu năng thời gian thực bằng Prometheus/Grafana — đưa hệ thống từ prototype nghiên cứu thành sản phẩm giáo dục hoàn chỉnh.

VI. References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey", *arXiv preprint arXiv:2312.10997*, 2023.

[3] S. Barnett et al., "Seven failure points when engineering a retrieval augmented generation system", *arXiv preprint arXiv:2401.05856*, 2024.

[4] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering", in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[5] L. Wang et al., "Text embeddings by weakly-supervised contrastive pre-training", in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 1112–1135.

[6] N. F. Liu et al., "Lost in the middle: How language models use long contexts", *arXiv preprint arXiv:2307.03172*, 2023.

[7] S. Xiao et al., "C-pack: Packaged resources to advance general chinese embedding", *arXiv preprint arXiv:2309.07597*, 2023.

[8] J. Chen et al., "Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation", *arXiv preprint arXiv:2402.03216*, 2024.

[9] Z. Ji et al., "Survey of hallucination in natural language generation", *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[10] T. Gao et al., "Enabling large language models to generate text with citations", in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 6465–6488.

[11] J. Wu et al., "Recursively summarizing books with human feedback", *arXiv preprint arXiv:2109.10862*, 2021.

[12] E. Kasneci et al., "Chatgpt for good? on opportunities and challenges of large language models for education", *Learning and Individual Differences*, vol. 103, p. 102 274, 2023.

[13] H. L. Roediger et al., "The power of testing memory: Basic research and implications for educational practice", *Perspectives on Psychological Science*, vol. 1, no. 3, pp. 181–210, 2006.

[14] S. Es et al., "Ragas: Automated evaluation of retrieval augmented generation", *arXiv preprint arXiv:2309.15217*, 2023.

[15] S. Robertson et al., "The probabilistic relevance framework: Bm25 and beyond", *Foundations and Trends® in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[16] G. V. Cormack et al., "Reciprocal rank fusion outperforms frontiers in internet retrieval", in *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, 2009, pp. 758–759.

[17] R. Rafailov et al., "Direct preference optimization: Your language model is secretly a reward model", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 53 728–53 741.

[18] E. J. Hu et al., "Lora: Low-rank adaptation of large language models", in *International Conference on Learning Representations (ICLR)*, 2022.

[19] T. Févry et al., "Colpali: Efficient document retrieval with vision language models", *arXiv preprint arXiv:2407.01449*, 2024.

[20] D. Edge et al., "From local to global: A graph rag approach to query-focused summarization", *arXiv preprint arXiv:2404.16130*, 2024.

[21] S.-Q. Yan et al., "Corrective retrieval augmented generation", *arXiv preprint arXiv:2401.15884*, 2024.